

```

    stop_timer(ack_expected % NR_BUFS); /* frame arrived intact */
    inc(ack_expected); /* advance lower edge of sender's window */
}
break;
case cksum_err:
    if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* damaged frame */
    break;
case timeout:
    send_frame(data, oldest_frame, frame_expected, out_buf); /* we timed out */
    break;
case ack_timeout:
    send_frame(ack, 0, frame_expected, out_buf); /* ack timer expired; send ack */
}
if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
}
}

```

图 3-21 (续)

非顺序接收引发了一些特殊问题, 这些问题对于那些按顺序接收帧的协议是不用考虑的。我们用一个例子就很容易说明麻烦之处。假设我们用 3 位序号, 那么发送方允许连续发送 7 个帧, 然后开始等待确认。刚开始时, 发送方和接收方的窗口如图 3-22 (a) 所示。现在发送方发出 0~6 号帧。接收方的窗口允许它接受任何序号落在 0~6 (含) 之间的帧。这 7 个帧全部正确地到达了, 所以接收方对它们进行确认, 并且向前移动它的窗口, 允许接收 7、0、1、2、3、4 或 5 号帧, 如图 3-22 (b) 所示。所有这 7 个缓冲区都标记为空。

此时, 灾难降临了, 闪电击中了电线杆子, 所有的确认都被摧毁。协议应该不管灾难是否发生都能正确工作。最终发送方超时, 并且重发 0 号帧。当这帧到达接收方时, 接收方检查它的序号, 看是否落在窗口中。不幸的是, 如图 3-22 (b) 所示, 0 号帧落在新窗口中, 所以它被当作新帧接受了。接收方同样返回 (捎带) 6 号帧确认, 因为 0~6 号帧都已经接收到了。

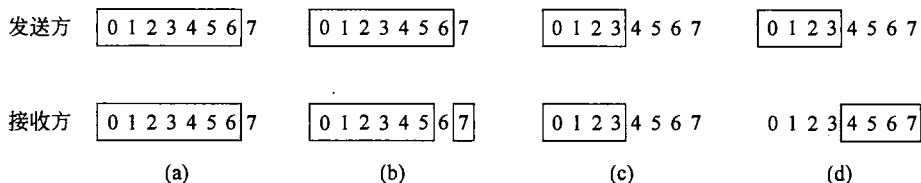


图 3-22

(a) 窗口大小为 7 的初始情形; (b) 发出 7 个帧并且接收 7 帧, 但尚未确认;
(c) 窗口大小为 4 的初始情形; (d) 发出 4 个帧并且接收 4 帧, 但尚未确认

发送方很高兴地得知所有它发出去的帧都已经正确地到达了, 所以它向前移动发送窗口, 并立即发送 7、0、1、2、3、4 和 5 号帧。7 号帧将被接收方接收, 并且它的数据包直接传递给网络层。紧接着, 接收方的数据链路层进行检查, 看它是否已经有一个有效的 0 号帧, 它发现确实已经有了 (即前面重发的 0 号帧), 然后把内嵌的数据包作为新的数据包传递给网络层。因此, 网络层得到了一个不正确的数据包。协议失败!

这个问题的本质在于: 当接收方向前移动它的窗口后, 新的有效序号范围与老的序号范围有重叠。因此, 后续的一批帧可能是重复的帧 (如果所有的确认都丢失了), 也可能是新的帧 (如果所有的确认都接收到了)。可怜接收方根本无法区分这两种情形。

解决这个难题的办法是确保接收方向前移动窗口之后,新窗口与老窗口的序号没有重叠。为了保证没有重叠,窗口的最大尺寸应该不超过序号空间的一半,如图 3-22 (c) 和图 3-22 (d) 所做的那样。如果用 3 位来表示序号,则序号范围为 0~7。在任何时候,应该只有 4 个未被确认的帧。按照这种方法,如果接收方已经接收了 0~3 号帧,并且向前移动了窗口,以便允许接收第 4~7 帧,那么,它可以明确地区分出后续的帧是重传帧(序号为 0~3),还是新帧(序列号为 4~7)。一般来说,协议 6 的窗口尺寸为 $(\text{MAX_SEQ}+1)/2$ 。

一个有意思的问题是:接收方必须拥有多少个缓冲区?无论如何,接收方不可能接受序号低于窗口下界的帧,也不可能接受序号高于窗口上界的帧。因此,所需要的缓冲区的数量等于窗口的大小,而不是序号的范围。在前面的 3 位序号例子中,只需要 4 个缓冲区就够了,编号为 0~3。当 i 号的帧到达时,它被放在 $(i \bmod 4)$ 号缓冲区中。请注意,虽然 i 和 $(i+4) \bmod 4$ 在“竞争”同一个缓冲区,但它们永远不会同时落在窗口内,因为如果那样的话,则窗口的尺寸至少为 5。

出于同样的原因,需要的计时器数量等同于缓冲区的数量,而不是序号空间的大小。实际上,每个缓冲区都有一个相关联的计时器。当计时器超时,缓冲区的内容就要被重传。

协议 6 还放宽了隐含假设,即信道的负载很重。我们在协议 5 中作了这个假设,因为协议要依赖反方向上的数据帧来捎带确认信息。如果逆向流量很轻,确认会被延缓很长一段时间,这将导致问题。在极端的情况下,如果在一个方向上有很大的流量,而另一个方向上根本没有流量,那么当发送方的窗口达到最大值后协议将被阻塞。

为了不受该假设的约束,当一个按正常次序发送的数据帧到达接收方之后,接收方通过 `start_ack_timer` 启动一个辅助的计时器。如果在计时器超时之前,没有出现需要发送的反向流量,则发送一个单独的确认帧。由该辅助计时器超时而导致的中断称为 `ack_timeout` 事件。在这样的处理方式下,缺少可以捎带确认的反向数据帧不再是一个问题,因此那些只存在单向数据流的场合依然可以使用该协议。只要一个辅助计时器就可以正常工作,如果该计时器在运行时又调用了 `start_ack_timer`,则该次调用不起作用。计时器不会被重置或者被扩展,因为它的作用是提供确认的某种最小速率。

辅助计时器的超时时间间隔应该明显短于与数据帧关联的计时器间隔,这点非常关键。这是确保下列情况的必要条件:即一个被正确接收的帧应该尽早地被确认,使得该帧的重传计时器不会超时因而不会重传该帧。

协议 6 采用了比协议 5 更加有效的策略来处理错误。当接收方有理由怀疑出现了错误时,它就给发送方返回一个否定确认 (NAK) 帧。这样的帧实际上是一个重传请求,在 NAK 中指定了要重传的帧。在两种情况下,接收方要特别注意:接收到一个受损的帧,或者到达的帧并非是自己所期望的(可能出现丢帧错误)。为了避免多次请求重传同一个丢失帧,接收方应该记录下对于某一帧是否已经发送过 NAK。在协议 6 中,如果对于 `frame_expected` 还没有发送过 NAK,则变量 `no_nak` 为 `true`。如果 NAK 被损坏了,或者丢失了,则不会有实质性的伤害,因为发送方最终会超时,无论如何它都会重传丢失的帧。如果一个 NAK 被发送出去之后丢失了,而接收方又收到一个错误的帧,则 `no_nak` 将为 `true`,并且辅助计时器将被启动。当该辅助计时器超时后,一个 ACK 帧将被发送出去,以便使发送方重新同步到接收方的当前状态。

在某些情形下,从一帧被发出去开始算起,到该帧经传播抵达目的地、在那里被处理,

然后它的确认被传回来，整个过程所需要的时间（几乎）是个常数。在这些情形下，发送方可以把它的计时器调得“紧”一些，让它恰好略微大于正常情况下从发送一帧到接收到其确认之间的时间间隔。此时 NAC 的作用就微乎其微了。

然而，在其他一些情况下这段时间的变化非常大。例如，如果反向流量零零散散，那么如果刚好有逆向流量则接收到帧延缓发送确认之前的这段时间会比较短；反之，如果没有逆向流量，则这段时间会很长。因此发送方必须做出选择，要么将计时器的间隔设置得比较小（其风险是不必要的重传），要么将它设置得比较大（发生错误之后长时间地空等）。两种选择都会浪费带宽。一般来说，当确认间隔的标准偏差与间隔本身相比非常大的时候，计时器应该设置得“松”一点。然后，NAK 可以加快丢失帧或者损坏帧的重传速度。

与超时和 NAK 紧密相关的一个问题是确定由哪一帧引发超时。在协议 5 中，引发超时的帧总是 `ack_expected` 指定的帧，因为它总是最早的那个帧。在协议 6 中，没有一种很具体的办法来确定谁来引发超时。假定已经发送了 0~4 号帧，这意味着未确认帧的列表是 01234，按照时间先后顺序排列。现在想象这样的情形：0 号帧超时，5 号帧（新帧）被发送出去，1 号帧超时，2 号帧超时，6 号帧（又一新帧）被发送出去。这时候，未确认帧的列表是 3405126，也是按照从最老的帧到最新的帧顺序排列。如果所有入境流量（即那些包含确认的帧）一下子全部被丢失，那么这 7 个未确认的帧将会依次超时。

为了避免使该例子过于复杂，我们没有显示计时器的管理过程。相反，我们只是假设在超时变量 `oldest_frame` 设置好之后，它就能指出哪一帧超时。

3.5 数据链路协议实例

在单个建筑物内，局域网被广泛用于互连主机，但大多数广域网的基础设施是以点到点方式建设的。我们将在第 4 章重点关注局域网，这里要考察的是那些出现在 Internet 两种常见情形下的数据链路协议，这些协议主要用在点到点的线路上。第一种情形是通过广域网中的 SONET 光纤链路发送数据包。例如，这些链路被广泛用于连接一个 ISP 网络中位于不同位置的路由器。

第二种情形是运行在 Internet 边缘的电话网络本地回路上的 ADSL 链路。这些链接把成千上百万的个人和企业连接到 Internet 上。

针对上述这些使用场景 Internet 需要点到点链路，还会用到拨号调制解调器、租用线路和线缆调制解调器等。所谓点到点协议（PPP，Point-to-Point Protocol）的标准协议就是使用这些链路来发送数据包。PPP 由 RFC1661 定义，并在 RFC1662 中得到进一步的阐述（Simpson, 1994a, 1994b）。SONET 和 ADSL 链路都采用了 PPP，但在使用方式上有所不同。

3.5.1 SONET 上的数据包

2.6.4 章节涉及的 SONET 是物理层协议，它最常被用在广域网的光纤链路上，这些光纤链路构成了通信网络的骨干网，其中包括电话系统。它提供了一个以定义良好速度运行

的比特流，比如 2.4 Gbps 的 OC-48 链路。比特流被组织成固定大小字节的有效载荷，不论是否有用户数据需要发送，每隔 125 微秒要发出一个比特流。

为了在这些链路上承载数据包，需要某种成帧机制，以便将偶尔出现的数据包从传输它们的连续比特流中区分出来。运行在 IP 路由器上的 PPP 就提供了这种运行机制，如图 3-23 所示。

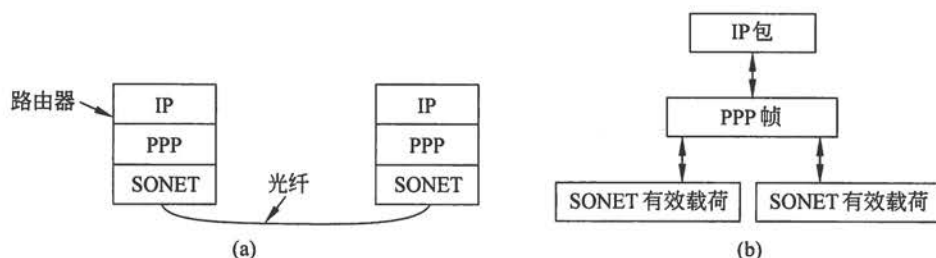


图 3-23 SONET 之上的数据包
(a) 协议栈；(b) 帧关系

PPP 功能包括处理错误检测链路的配置、支持多种协议、允许身份认证等。它是一个早期简化协议的改进，那个协议称为串行线路 Internet 协议 (SLIP, Serial Line Internet Protocol)。伴随着一组广泛选项，PPP 提供了 3 个主要特性：

(1) 一种成帧方法。它可以毫无歧义地区分出一帧的结束和下一帧的开始。

(2) 一个链路控制协议。它可用于启动线路、测试线路、协商参数，以及当线路不再需要时温和地关闭线路。该协议称为链路控制协议 (LCP, Link Control Protocol)。

(3) 一种协商网络层选项的方式。协商方式独立于网络层协议。所选择的方法是针对每一种支持的网络层都有一个不同的网络控制协议 (NCP, Network Control Protocol)。

因为没有必要重新发明轮子，所以 PPP 帧格式的选择酷似 HDLC 帧格式。HDLC 是高级数据链路控制协议 (High-level Data Link Control)，是一个早期被广泛使用的家庭协议实例。

PPP 和 HDLC 之间的主要区别在于：PPP 是面向字节而不是面向比特的。特别是 PPP 使用字节填充技术，所有帧的长度均是字节的整数倍。HDLC 协议则使用比特填充技术，允许帧的长度不是字节的倍数，例如 30.25 字节。

然而，实际上还有第二个主要区别。HDLC 协议提供了可靠的数据传输，所采用的方式正是我们已熟悉的滑动窗口、确认和超时机制等。PPP 也可以在诸如无线网络等嘈杂的环境里提供可靠传输，具体细节由 RFC1663 定义。然而，实际上很少这样做。相反，Internet 几乎都是采用一种“无编号模式”来提供无连接无确认的服务。

PPP 帧格式如图 3-24 所示。所有的 PPP 帧都从标准的 HDLC 标志字节 0x7E(01111110) 开始。标志字节如果出现在 Payload 字段，则要用转义字节 0x7D 去填充；然后将紧跟在后面的那个字节与 0x20 进行 XOR 操作，如此转义使得第 6 位比特反转。例如 0x7D 0x5E 是标志字节 0x7E 的转义序列。这意味着只需要简单扫描 0x7E 就能找出帧的开始和结束之处，因为这个字节不可能出现在其他地方。接收到一个帧后要去掉填充字节。具体做法是，扫描搜索 0x7D，发现后立即删除；然后用 0x20 对紧跟在后面的那个字节进行 XOR 操作。此外，两个帧之间只需要一个标志字节。当链路上没有帧在发送时，可以用多个标志字节填充。

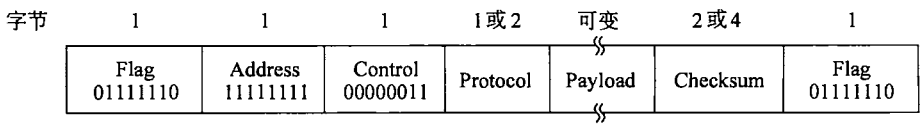


图 3-24 无编号模式操作下的完整 PPP 帧格式

紧跟在帧开始处标记字节后面出现的是 Address（地址）字段。这个字段总是被设置为二进制值 11111111，表示所有站点都应该接受该帧。使用这个值可避免如何为数据链路层分配地址这样的问题。

Address 字段后面是 Control（控制）字段，其默认值是 00000011。此值表示一个无编号帧。

因为 Address 和 Control 字段总是取默认配置的常数，因此 LCP 提供了某种必要的机制，允许通信双方就是否省略这两个字段进行协商，去掉的话可以为每帧节省 2 个字节的

空间。
PPP 的第四个字段是 Protocol（协议）字段。它的任务是通告 Payload 字段中包含了什么类型的数据包。以 0 开始的编码定义为 IP 版本 4、IP 版本 6 以及其他可能用到的网络层协议，比如 IPX 和 AppleTalk。以 1 开始的编码被用于 PPP 配置协议，包括 LCP 和针对每个网络层协议而设置的不同 NCP。Protocol 字段的默认大小为 2 个字节，但它可以通过 LCP 协商减少到 1 个字节。协议设计师们也许过于谨慎了，认为有一天可能使用超过 256 个协议。

Payload（有效载荷）字段是可变长度的，最高可达某个协商的最大值。如果在链路建立时没有通过 LCP 协议进行协商，则采用默认长度 1500 字节。如果需要，在有效载荷后填充字节以便满足帧的长度要求。

Payload 字段后是 Checksum（校验和）字段，它通常占 2 个字节，但可以协商使用 4 个字节的校验和。事实上，4 字节的校验和与 32 位的 CRC 相同，其生成多项式就是 3.3.2 章节末尾给出的那个多项式。2 字节的校验和也是一个工业标准 CRC。

PPP 是一个成帧机制，它可以在多种类型的物理层上承载多种协议的数据包。为了在 SONET 上使用 PPP，RFC2615 列出了一些可用的选择（Malis 和 Simpson，1999）。因为这是检测物理层、数据链路层和网络层传输错误的主要手段，因此采用了 4 字节的校验和。它还建议不要压缩 Address、Control 和 Protocol 字段，因为 SONET 链路的运行速度已经达到很高了。

PPP 还有一个不同寻常的特点。PPP 的有效载荷在插入到 SONET 的有效载荷之前先进行扰码操作（正如 2.5.1 所描述的那样）。用长伪随机序列对有效载荷进行扰码 XOR 之后再传送出去。问题在于为了保持同步 SONET 比特流，必须频繁地进行比特跳变。这些跳变很自然地与语音信号的变化结合在一起，但在数据通信中用户选择发送的信息和数据包可能包含一长串的 0。有了扰频技术，用户因为发送一长串 0 而导致问题的可能性降到极低。

在通过 SONET 线路发送 PPP 帧之前，必须建立和配置 PPP 链路。PPP 链路的启动、使用和关闭的一系列阶段如图 3-25 所示。

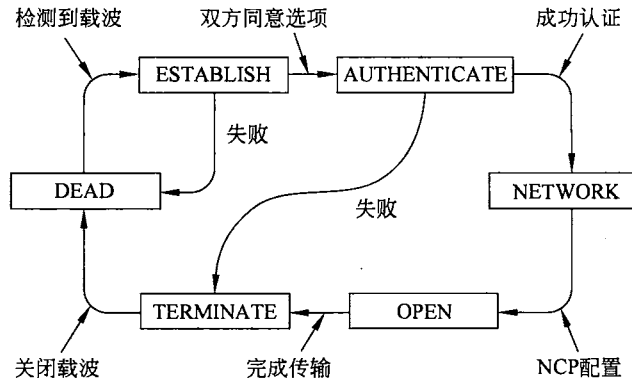


图 3-25 PPP 链路建立到释放的状态转换图

链路的初始状态为 DEAD（死），这意味着不存在物理层连接。当物理连接被建立起来，链路转移到 ESTABLISH（建立）状态。此时，PPP 对等实体交换一系列的 LCP 报文进行上面所说的那些 PPP 选项的协商，这些 LCP 报文放在 PPP 帧的 Payload 字段。初始发起连接的实体提出自己的选项请求，对等实体可以部分或者全部接受，甚至全部拒绝，同时它也可以提出自己的选项要求。

如果 LCP 选项协商成功，链路状态进入 AUTHENTICATE（认证）状态。现在，如果需要，双方可以互相检查对方的身份。如果认证成功，则链路进入 NETWORK（网络）状态，通过发送一系列的 NCP 包来配置网络层参数。NCP 协议很难一概而论，因为每个协议特定于实际采用的某个网络层协议，允许执行针对特定于该网络层协议的配置请求。例如，对于 IP 协议而言，为链路的两端分配 IP 地址是最重要的可能操作。

一旦进入 OPEN（打开）状态，双方就可以进行数据传输。正是在这个状态下，IP 数据包被承载在 PPP 帧中通过 SONET 线路传输。当完成数据传输后，链路进入 TERMINATE（终止）状态；当物理层连接被舍弃后从这里回到 DEAD 状态。

3.5.2 对称数字用户线

ADSL 以 Mbps 速率将百万计的家庭用户连接到 Internet 上，并且使用的是与普通老式电话服务相同的本地回路。在 2.5.3 节我们介绍了如何把一种称为 DSL 调制解调器的设备安置在家庭一端。它通过本地回路发出的数据比特到达一个称为 DSLAM（DSL 接入复用器，发音为“dee-slam”）的设备，该设备安置在电话公司端局。现在，我们将更详细地探讨如何在 ADSL 链路上承载数据包。

ADSL 使用的协议和设备的概貌如图 3-26 所示。不同的网络可以部署不同的协议，所以我们选择了最流行的场景。在家里，比如 PC 机那样的计算机使用以太网链路把 IP 数据包发送到 DSL 调制解调器；然后 DSL 调制解调器通过本地回路把 IP 数据包发送到 DSLAM，发送数据包所用的协议就是我们将要学习的协议。在 DSLAM 设备（或连接到它路由器，视具体实施而定）上 IP 数据包被提取出来，并被注入到 ISP 网络，因此它们最终能到达 Internet 上的任何目的地。

在图 3-26 中，ADSL 链路之上的协议底部是 ADSL 物理层。它们基于称为正交频分复

用（也称为离散多音）的数字调制方案，就像我们在 2.5.3 节看到的那样。接近协议栈顶部，恰好位于 IP 网络层正下方的是 PPP。该协议与我们刚刚考察过在 SONET 上传输数据包的 PPP 相同。它以同样的方式来建立和配置链路，以及运载 IP 数据包。

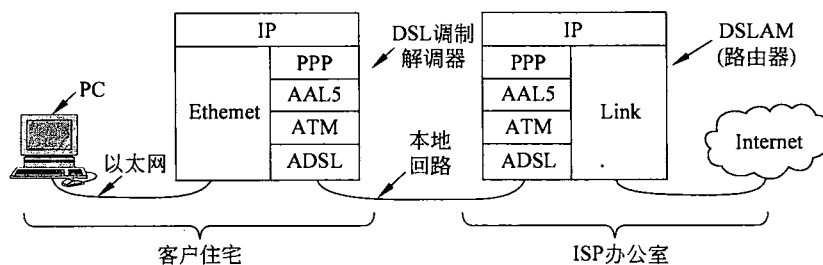


图 3-26 ADSL 协议栈

在 ADSL 和 PPP 两者之间的是 ATM 和 AAL5。这些是我们以前从未见过的新协议。异步传输模式（ATM, Asynchronous Transfer Mode）早在 20 世纪 90 年代初就被设计了出来，并且以令人难以置信的炒作方式公布于众。它承诺的网络技术可以解决世界上的电信问题，它把语音、数据、有线电视、电报、信鸽、串起来的罐头、击鼓，以及其他所有一切合并成一个综合系统，声称能为每个人做每件事。但事实并非如此。在很大程度上，ATM 的问题类似于我们介绍的 OSI 协议，也就是说，错误的时机、错误的技术、错误的实施以及错误的政治。不过，ATM 的命运远好于 OSI。虽然它并没有在世界各地流行起来，但它仍然被广泛应用在合适的领域，包括诸如 DSL 那样的宽带接入线路以及电话网络内部的广域网链路。

ATM 是一种链路层，它的传输基于固定长度的信息信元（cell）。其名称中的“异步”意味着这些信元并不总是以连续的方式发送，这与 SONET 在同步线路上发送的方式不同。只有当出现需要运载的信息时，才发送信元。ATM 是一种面向连接的技术。每个信元在它的头部带有虚电路（virtual circuit）标识符，交换设备根据此标识符沿着连接建立的路径转发信元。

每个信元 53 字节长，由一个 48 字节的有效载荷以及一个 5 字节的头组成。利用这些小信元，ATM 可以为不同用户灵活划分物理层链路的带宽。这种能力对网络应用非常有用，例如，在同一条链路上发送语音信息和数据信息，不会因为出现很长的数据包而导致声音样本值的延迟变化过大。信元长度的与众不同选择（例如，相比之下，更自然的选择是 2 的幂次方）恰好说明 ATM 的设计是多么的政治化。选择有效载荷的长度为 48 字节正是解决欧洲和美国分歧的一个妥协，欧洲希望信元取 32 字节长，而美国方面则希望信元取 64 字节长。有关 ATM 的简要概述请参考（Siu 和 Jain, 1995）。

为了在 ATM 网络上发送数据，需要将数据映射成一系列的信元。这个映射由 ATM 适配层完成，映射过程称为分段（segmentation）和重组（reassemble）。针对不同的服务定义了几个适配层，从周期性的声音样本到数据包数据。其中一个主要用于数据包数据的适配层是 ATM 适应层 5（AAL5, ATM Adaptation Layer 5）。

一个 AAL5 帧如图 3-27 所示。除了帧头，它还有一个帧尾，给出了帧的长度和用于错误检测的 4 字节 CRC。很自然，这里的 CRC 与 PPP 和诸如以太网那样的 IEEE 802 局域网使用的相同。（Wang 和 Crowcroft, 1992）已表明该 CRC 强大到足以检测出非传统错误（诸

如信元重新排序)。除了有效载荷外，AAL5 帧还需要被填充。填充的目的是使得帧的总长度是 48 字节的倍数，以便帧被均匀地划分成多个信元。这里不需要地址，因为每个信元携带的虚电路标识将引导它到达正确的目的地。

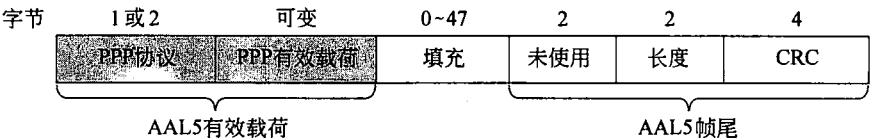


图 3-27 运载 PPP 数据的 AAL5 帧

现在，我们已经简单描述了 ATM，但仅涉及在 ADSL 的情况下 PPP 如何使用 ATM。这项工作由另一个称为 ATM 上的 PPP (PPPoA, PPP over ATM) 标准完成。这个标准不是真正意义上的协议 (所以没出现在图 3-26 中)，但详细说明了 PPP 和 AAL5 帧如何工作的，该标准由 RFC2364 描述 (Gross 等，1998)。

只有 PPP 协议和有效载荷字段被放置在 AAL5 的有效载荷，如图 3-27 所示。协议字段告诉远端 DSLAM 有效载荷里包含的是一个 IP 数据包，或是另一种协议的数据包 (比如 LCP 协议)。远端当然知道信元包含了 PPP 信息，因为 ATM 虚电路就是为这个目的而建立的。

在 AAL5 帧内，PPP 的成帧功能并不是必需的，因为在这里起不到任何作用；ATM 和 AAL5 早已提供了成帧的功能。更多的成帧考虑将毫无价值。同样，PPP 的 CRC 也没有必要，因为 AAL5 已经包括了相同的 CRC。这个错误检测机制补充了 ADSL 的物理层功能，ADSL 物理层采用 Reed-Solomon 码来检测错误，同时还用 1 字节的 CRC 来检测遗漏的任何错误。该方案具有比在 SONET 上发送数据包更为复杂的错误恢复机制，因为 ADSL 是一种非常嘈杂的信道。

3.6 本章总结

数据链路层的任务是将物理层提供的原始比特流转换成由网络层使用的帧流。链路层为这样的帧流提供不同程度的可靠性，范围从无连接无确认的服务到可靠的面向连接服务不等。

链路层采用的成帧方法各种各样，包括字节计数、字节填充和比特填充。数据链路协议提供了差错控制机制来检测或纠正传输受损的帧，以及重新传输丢失的帧。为了防止快速发送方淹没慢速接收方，数据链路协议还提供了流量控制机制。滑动窗口机制被广泛用来以一种简单方式集成差错控制和流量控制两大机制。当窗口大小为 1 个数据包时，则协议是停-等式的。

纠错和检错码使用不同的数学技术把冗余信息添加到消息中。卷积码和里德所罗门码被广泛用于纠错，低密度校验码越来越受到欢迎。实际使用的检错码包括循环冗余校验和校验和两种。所有这些编码方法不仅可被应用在链路层，而且也可用在物理层和更高的层次。

我们考察了一系列协议，这些协议在更现实的假设下通过确认和重传，或者 ARQ (自

动重复请求)机制为上层提供了一个可靠的链路层。从一个无错误的环境开始,即接收方可以处理传送给它的任何帧,我们引出了流量控制,然后是带有序号的差错控制和停-等式算法。然后,我们使用滑动窗口算法允许双向通信,并引出捎带确认的概念。最后给出两个协议把多个帧的传输管道化,以此来防止发送方被一个有着漫长传播延迟的链路所阻塞。接收方可以丢弃所有乱序的帧,或者为了获得更大的带宽效率而缓冲这些乱序帧,并且给发送方反馈否定确认。前一种策略是回退 n 协议,后一种策略是选择重传协议。

Internet 使用 PPP 作为点到点线路上的主要数据链路协议。PPP 协议提供了无连接的无确认服务,使用标志字节区分帧的边界,至于错误检测则采用 CRC。该协议通常被用在运载数据包的一系列链路上,包括广域网中的 SONET 链路和家庭 ADSL 链路。

习 题

1. 一个上层数据包被分成 10 个帧,每一帧有 80% 的机会无损地到达目的地。如果数据链路协议没有提供错误控制,试问,该报文平均需要发送多少次才能完整地到达接收方?
2. 数据链路协议使用了下面的字符编码:
A: 01000111; B: 11100011; FLAG: 01111110; ESC: 11100000
为了传输一个包含 4 个字符的帧: A B ESC FLAG,试问使用下面的成帧方法时所发送的比特序列(用二进制表达)是什么?
(a) 字节计数。
(b) 字节填充的标志字节。
(c) 比特填充的首尾标志字节。
3. 一个数据流中出现了这样的数据段: A B ESC C ESC FLAG FLAG D,假设采用本章介绍的字节填充算法,试问经过填充之后的输出是什么?
4. 试问字节填充算法的最大开销是多少?
5. 你的一个同学 Scrooge 指出每一帧的结束处用一个标志字节而下一帧的开始处又用另一个标志字节,这种做法非常浪费空间。用一个标志字节也可以完成同样的任务,这样可以节省下来一个字节。试问你同意这种观点吗?
6. 需要在数据链路层上被发送一个比特串: 011110111110111110。试问,经过比特填充之后实际被发送出去的是什么?
7. 试问在什么样的环境下,一个开环协议(比如海明码)有可能比本章通篇所讨论的反馈类协议更加适合?
8. 为了提供比单个奇偶位更强的可靠性,一种检错编码方案如下:用一个奇偶位来检查所有奇数序号的位,用另一个奇偶位来检查所有偶数序号的位。请问这种编码方案的海明距离是多少?
9. 假设使用海明码来传输 16 位的报文。试问,需要多少个校验位才能确保接收方能同时检测并纠正单个比特错误?对于报文 1101001100110101,试给出传输的比特模式。假设在海明码中使用了偶校验。
10. 接收方收到一个 12 位的海明码,其 16 进制值为 0xE4F。试问该码的原始值是多少?

假设至多发生了一位错。

11. 检测错误的一种方法是按 n 行、每行 k 位来传输数据，并且在每行和每列加上奇偶位。其中最右下角是一个校验其所在行和列的奇偶位。试问这种方案能检测出所有的 1 位错吗？2 位错误呢？3 位错误呢？请说明这种方案无法检测出某些 4 位错误。
12. 假设数据以块形式传输，每块大小 1000 比特。试问，在什么样的最大错误率下，错误检测和重传机制（每块 1 个校验位）比使用海明码更好？假设比特错误相互独立，并且在重传过程中不会发生比特错误。
13. 一个具有 n 行 k 列的块使用水平和垂直奇偶校验位进行差错检测。假设正好有 4 位由于传输错误被反转。请推导出该错误无法被检测出来的概率表达式。
14. 利用如图 3-7 所示的卷积码，当输入序列是 10101010（从左到右）并且内部状态初始化为全零时，试问，输出序列是什么？
15. 假设使用 Internet 校验和（4 位字）来发送一个消息 1001110010100011。试问校验和的值是什么？
16. x^7+x^5+1 被生成多项式 x^3+1 除，试问，所得余数是什么？
17. 使用本章介绍的标准 CRC 方法传输比特流 10011101。生成多项式为 x^3+1 。试问实际传输的位串是什么？假设左边开始的第三个比特在传输过程中变反了。请说明这个错误可以在接收方被检测出来。给出一个该比特流传输错误的实例，使得接收方无法检测出该错误。
18. 发送一个长度为 1024 位的消息，其中包含 992 个数据位和 32 位 CRC 校验位。CRC 计算采用了 IEEE 802 标准，即 32 阶的 CRC 多项式。对于下面每种情况，说明在信息传输中出现的错误能否被接收方检测出来：
 - (a) 只有一位错误。
 - (b) 有 2 个孤立的一位错误。
 - (c) 有 18 个孤立的一位错误。
 - (d) 有 47 个孤立的一位错误。
 - (e) 有一个长度为 24 位的突发错误。
 - (f) 有一个长度为 35 位的突发错误。
19. 在 3.3.3 节讨论 ARQ 协议时，概述了一种场景，由于确认帧的丢失导致接收方接收了两个相同的帧。试问，如果不会出现丢帧（消息或者确认），接收方是否还有可能收到同一帧的多个副本？
20. 考虑一个具有 4 kbps 速率和 20 毫秒传输延迟的信道。试问帧的大小在什么范围内，停-等式协议才能获得至少 50% 的效率？
21. 在协议 3 中，当发送方的计时器已经在运行时，它还有可能启动该计时器吗？如果可能，试问这种情况是如何发生的？如果不可能，试问为什么不可能？
22. 使用协议 5 在一条 3000 千米长的 T1 中继线上传输 64 字节的帧。如果信号的传播速度为 6 微秒/千米，试问序号应该有多少位？
23. 想象一个滑动窗口协议，它的序号占用的位数相当多，使得序号几乎永远不会回转。试问 4 个窗口边界和窗口大小之间必须满足什么样的关系？假设这里窗口的大小固定不变，并且发送方和接收方的窗口大小相同。

24. 在协议 5 中, 如果 between 过程检查的条件是 $a \leq b \leq c$, 而不是 $a \leq b < c$, 试问这对于协议的正确性和效率有影响吗? 请解释你的答案。
25. 在协议 6 中, 当一个数据帧到达时, 需要检查它的序号是否不同于期望的序号, 并且 no_nak 为真。如果这两个条件都成立, 则发送一个 NAK; 否则, 启动辅助定时器。假定 else 子句被省略掉, 试问这种改变会影响协议的正确性吗?
26. 假设将协议 6 中接近尾部的内含三条语句的 while 循环去掉, 试问这样会影响协议的正确性吗? 或者只是仅仅影响了协议的性能? 请解释你的答案。
27. 地球到一个遥远行星的距离大约是 9×10^{10} 米。如果采用停-等式协议在一条 64 Mbps 的点到点链路上传输帧, 试问信道的利用率是多少? 假设帧的大小为 32 KB, 光的速度是 3×10^8 m/s。
28. 在前面的问题中, 假设用滑动窗口协议来代替停-等式协议。试问多大的发送窗口才能使得链路利用率为 100%? 发送方和接收方的协议处理时间可以忽略不计。
29. 在协议 6 中, frame_arrival 代码中有一部分是用来处理 NAK 的。如果入境帧是一个 NAK, 并且另一个条件也满足, 则这部分代码会被调用。请给出一个场景, 在此场景下另一个条件非常关键。
30. 考虑在一条 1 Mbps 的完美线路(即无差错)上使用协议 6。帧的最大长度为 1000 位。每过 1 秒产生一个新数据包。超时间隔为 10 毫秒。如果取消特殊的确认计时器, 那么就会发生不必要的超时事件。试问, 平均报文要被传输多少次?
31. 在协议 6 中, $MAX_SEQ = 2^n - 1$ 。虽然这种情况显然是希望尽可能利用头部空间, 但我们无法证明这个条件是基本的。试问, 当 $MAX_SEQ = 4$ 时协议也能够正确工作吗?
32. 利用地球同步卫星在一个 1 Mbps 的信道上发送长度为 1000 位的帧, 该信道的传播延迟为 270 毫秒。确认总是被捎带在数据帧中。帧头非常短, 序号使用了 3 位。试问, 在下面的协议中, 可获得的最大信道利用率是多少?
 - (a) 停等式?
 - (b) 协议 5?
 - (c) 协议 6?
33. 在一个负载很重的 50 kbps 卫星信道上使用协议 6, 数据帧包含长度为 40 位的头和长度为 3960 位的数据, 试问浪费的带宽开销(帧头和重传)占多少比例? 假设从地球到卫星的信号传播时间为 270 毫秒。ACK 帧永远不会发生。NAK 帧长 40 位。数据帧的错误率是 1%, NAK 帧的错误率忽略不计。序号占 8 位。
34. 考虑在一个无错的 64kbps 卫星信道上单向发送 512 字节长的数据帧, 来自另一个方向反馈的确认帧非常短。对于窗口大小为 1、7、15 和 127 的情形, 试问最大的吞吐量分别是多少? 从地球到卫星的传播时间为 270 毫秒。
35. 在一条 100 千米长的线缆上运行 T1 数据速率。线缆的传播速度是真空中光速的 2/3。试问线缆中可以容纳多少位?
36. PPP 使用字节填充而不是比特填充, 这样做的目的是防止有效载荷字段偶尔出现的标志字节造成的混乱。请至少给出一个理由说明 PPP 为什么这么做。
37. 试问, 使用 PPP 发送一个 IP 数据包的最低开销是多少? 如果只计算 PPP 自身引入的开销, 而不计 IP 头开销, 试问最大开销又是多少?

38. 在本地回路上使用 ADSL 协议栈来发送一个长为 100 个字节的 IP 数据包。试问一共发出去多少个 ATM 信元？请简要描述这些信元的内容。
39. 本实验练习的目标是用本章描述的标准 CRC 算法实现一个错误检测机制。编写两个程序：generator 和 verifier。generator 程序从标准输入读取一行 ASCII 文本，该文本包含由 0 和 1 组成的 n 位消息。第二行是个 k 位多项式，也是以 ASCII 码表示。程序输出到标准输出设备上的是一行 ASCII 码，由 $n+k$ 个 0 和 1 组成，表示被发送的消息。然后，它输出多项式，就像它输入的那样。verifier 程序读取 generator 程序的输出，并输出一条消息指示正确与否。最后，再写一个程序 alter，它根据参数（从最左边开始 1 的比特数）反转第一行中的比特 1，但正确复制两行中的其余部分。通过键入：

```
generator <file | verifier
```

你应该能看到正确的消息，但键入：

```
generator <file | alter arg | verifier
```

你只能得到错误的消息。

第 4 章 介质访问控制子层

网络链路可以分成两大类：使用点到点连接和使用广播信道。我们在第 2 章学习了点到点链路，本章将讨论广播网络和相应的协议。

在任何一个广播网络中，关键的问题是当多方竞争信道的使用权时如何确定谁可以使用信道。为了把这个问题表述得更加清晰，我们用一个电话会议的场景来模拟说明：现在有 6 个人分别守在 6 部不同的电话机旁，这些电话相互之间都有连接，所以每个人都可以听到其他人说话，也可以对其他人讲话。当一个人停止说话时，很可能马上有两个或者更多个人开始说话，从而导致交流的一片混乱。而在面对面坐着的会议上，这种混乱局面可以通过外部途径得到解决，比如，让与会者通过举手的方式请求获得发言权。当只有一条信道可供使用时，确定下一个使用者的确非常困难。解决这个问题的许多协议已众所周知。这些协议组成了本章的内容。在有些文献中，广播信道有时候也称为多路访问信道（multiaccess channel）或者随机访问信道（random access channel）。

用来确定多路访问信道下一个使用者的协议属于数据链路层的一个子层，该层称为介质访问控制（MAC, Medium Access Control）子层。在 LAN 中，MAC 子层显得尤为重要，特别是在无线局域网中，因为无线本质上就是广播信道。相反，WAN 则使用点到点链路，当然，卫星网络除外。因为多路访问信道和 LAN 如此紧密相关，所以，在本章我们将从总体上讨论 LAN，其中也包括一些从严格意义上讲并不属于 MAC 子层的内容，但是总的主題还是关于信道控制。

技术上，MAC 子层位于数据链路层底部，所以，逻辑上我们应该在第 3 章讨论所有点到点协议之前学习 MAC 子层。然而，对于大多数人来说，在很好地理解了只有两方参与的协议之后，再来理解涉及多方协同的协议要容易得多。正是基于这样的原因，我们才稍微偏离了本书自底向上的严格顺序。

4.1 信道分配问题

本章的主题是如何在竞争用户之间分配单个广播信道。信道可以是一个地理区域内的一部分无线频谱，也可以是连接着多个节点的单根电缆或者光纤，这都无关紧要。在这两种情况下，信道把每个用户与所有其他用户连接在一起，任何正在使用信道的用户和其他也想使用该信道的用户会相互干扰。

我们首先考察静态分配方案在突发流量情况下的缺点；然后，我们给出一些关键假设，这些假设在后面讨论动态分配方案的模型时要用到。

4.1.1 静态信道分配

在多个竞争用户之间分配单个信道的传统做法是把信道容量拆开分给多个用户使用，

具体方法可以采用我们在 2.5 节中讨论的某种多路复用技术，比如 FDM（频分多路复用），就像多个竞争用户多路复用电话中继线一样。如果总共有 N 个用户，则整个带宽被分成 N 等份，每个用户获得一份。由于每个用户都有各自专用的频段，所以用户之间不会发生干扰。当用户数量比较少且固定不变时，如果每个用户都有稳定的流量或者负载繁重，这种信道分割是一种简单而有效的分配机制。FM 无线电广播就是一个无线信道的多路复用例子。每个电台获得 FM 波段一部分的使用权，大部分时间用该波段广播自己的信号。

然而，当发送方的数量非常多而且经常不断变化，或者流量呈现突发性，FDM 就存在一些问题。如果整个频谱被切割成 N 份，并且当前只有很少的用户（比 N 少得多）需要进行通信，那么大量宝贵的频谱将被浪费掉。但是，如果希望进行通信的用户数超过了 N 个，则有些用户将因带宽不够而遭到拒绝；即使有些已经被分配了频段的用户什么都不发送或者什么都不接收，也无法将它们的频段让给其他需要的用户使用。

然而，即使我们假设用户数量能够维持在 N 个固定不变，将单个信道划分成多个静态子信道的做法本质上也是低效的。基本问题在于，当有些用户停止通信时，他们的带宽实际上就被白白浪费了。他们自己不使用这些带宽，其他用户也不允许使用。静态分配方案很难适应大多数计算机系统，在计算机系统中数据流量表现出极端的突发性，通常峰值流量与平均流量之比能达到 1000:1。因此，大多数信道在多数时候是空闲的。

静态 FDM 如此差的性能通过一个简单的排队理论计算很容易看得更清楚。我们考虑在一个容量为 C bps 的信道上发送一帧所需要的平均时延 T 。假设，随机到达帧的平均到达率为 λ 帧/秒，帧的长度可变，其均值为每帧 $1/\mu$ 位。利用这些参数，可以算出信道的服务率为 μC 帧/秒。标准排队理论的结果是：

$$T = \frac{1}{\mu C - \lambda}$$

（很好奇的是这个结果是一个 M/M/1 排队。它要求帧到达的时间差和帧的长度符合遵循指数分布的随机过程，或者等价于泊松过程的结果。）

在我们的例子中，如果 C 为 100 Mbps，平均帧长 $1/\mu$ 等于 10 000 位，帧的到达率为 5000 帧/秒，则 $T=200$ 微秒。请注意，如果我们忽略排队延迟，只是问“在一个 100 Mbps 的网络上发送一个 10 000 位的帧需要多长时间”，那么我们将得到（不正确的）答案 100 微秒。这样的结果只有当不存在信道竞争时才成立。

现在我们将单个信道分成 N 个独立的子信道，每个子信道的容量为 C/N bps。现在，每个子信道的平均到达率变成 λ/N 。重新计算 T ，我们得到：

$$T_N = \frac{1}{\mu(C/N) - (\lambda/N)} = \frac{N}{\mu C - \lambda} = NT \quad (4-1)$$

如果所有帧都能很神奇地排在一个大的中心队列，那么划分信道后单个信道的平均延迟比不分的情况差 N 倍。同样的结论适用在银行。在一家银行的大堂摆满了 ATM 机，设立单个到达所有这些 ATM 机的队列比为每台 ATM 机器设立一个单独队列的效果要好。

适用于 FDM 的结论同样也适用于其他静态划分信道的方法。如果我们打算使用时分多路复用（TDM）技术为每个用户固定分配每 N 个时间槽，当一个用户没有使用分配给他的时间槽，那么该时间槽就会空闲。如果我们从物理上将网络分割开，则存在同样的问题。继续引用前面的例子，如果我们用 10 个 10 Mbps 的网络来代替一个 100 Mbps 的网络，并

且为每个用户固定分配其中的一个网络,则平均延迟将从 200 微秒跳跃到 2 毫秒。

既然所有的传统静态信道分配方法都不适应突发性的流量,我们现在就来研究动态的信道分配方法。

4.1.2 动态信道分配的假设

本章将介绍许多种信道分配方案,在讨论第一种方案之前,有必要认真地形式化信道分配问题。在这方面做的所有工作都是以下面 5 个关键假设为基础的。

(1) **流量独立 (independent traffic)**。该模型是由 N 个独立的站 (比如计算机、电话) 组成的,每个站都有一个程序或者用户产生要传输的帧。在长度为 Δt 的间隔内,期望产生的帧数是 $\lambda \Delta t$,这里 λ 为常数 (新帧的到达率)。一旦生成出一帧,则站就被阻塞,直到该帧被成功地发送出去。

(2) **单信道 (Single Channel)**。所有的通信都用这一个信道。所有的站可以在该信道上传输数据,也可以从该信道接收数据。所有站的能力都相同,尽管协议可能为站分配不同的角色 (比如,优先级)。

(3) **冲突可观察 (observable Collision)**。如果两帧同时传输,则它们在时间上就重叠,由此产生的信号是混乱的,这种情况称为**冲突 (collision)**。所有的站都能够检测到冲突事件的发生。冲突的帧必须在以后再次被发送。除了因冲突而产生错误外,不会再有其他的错误。

(4) **时间连续或分槽 (Continuous or slotted time)**。时间可以假设是连续的,即在任何时刻都可以开始传输帧。另一种选择是把时间分槽或者分成离散的间隔 (称为时间槽)。帧的传输只能从某个时间槽的起始点开始。一个时间槽可能包含 0、1 或者多个帧,分别对应于空闲的时间槽、一次成功发送,或者一次冲突。

(5) **载波侦听或不听 (Carrier Sense or no carrier sense)**。有了载波侦听的假设,一个站在试图用信道之前就能知道该信道当前是否正被使用。如果信道侦听结果是忙,则没有一个站会再去试图使用该信道。如果没有载波侦听,站就无法在使用信道之前侦听信道,它们只能盲目地传输,以后再判断这次传输是否成功。

下面依次对这些假设进行讨论。第一个假设意味着帧的到达是独立的,无论是跨越多个站还是某个特定的站,帧的产生不可预测但以恒定的速率产生帧。其实,正如众所周知的那样,针对网络流量的这种假设并不是个很好的模型,因为数据包在一个时间尺度范围内的到达呈现突发性 (Paxson 和 Floyd, 1995; Leland 等, 1994)。尽管如此,随着被频繁地应用,泊松模型因其在数学上易于处理而成为一个很有用的工具。它们能帮助我们分析协议,理解协议在大致操作范围内的性能如何变化,以及如何与其他协议比较。

单信道的假设是该模型的核心。除了这个信道外,没有任何外部途径可以通信。这些站不可能举起手来请求老师准许发言,因此我们必须拿出更好的解决方案。

其余的三个假设依赖于该系统的工程。当我们考察一个特定的协议,我们会说这些假设是成立的。

冲突假设是最基本的。当站发送时需要一些方法来检测是否发生了冲突,由此决定重传帧而不是任由那些帧被丢失。对于有线信道,节点的硬件可设计成一边发送一边检测冲

突；然后，如果发生了冲突，该站可提前终止传输，以免浪费信道容量。这种检测对于无线信道很难做到，所以冲突的检测通常被推迟到确信没有出现预期的确认帧这样一个既成事实之后。同时，卷入冲突的一些帧也有可能最终被成功接收，这主要取决于具体的信号强度和接收硬件的能力。不过，这种情况很少见，因此我们假设发生冲突后所有涉及的帧都被丢失。我们还将看到一些协议被设计成旨在防止冲突的发生。

对时间给出两种不同假设的理由在于时间槽可用来改善协议性能。然而，这要求所有站遵循一个主时钟或者它们的行动与其他站同步，才能将时间分为离散的间隔。因此，它并不总是可用的。我们对这两类时间都进行了讨论和分析。对于一个给定的系统，它只能支持一种时间假设。

类似地，一个网络可能具有载波侦听功能，也可能没有载波侦听功能。有线网络通常具有载波侦听功能。然而，无线网络不能有效地使用它，因为并不是每个站都在其他各站的无线电广播范围内。同样，在站不能直接和其他各站通信的一些其他设置中，载波侦听也不可用，例如线缆调制解调器，站必须通过线缆头端才能通信。注意，这个意义上的载波（carrier）一词是指信道上的信号，与公共运营商（例如，电话公司）没有任何关系，说起公共运营商可以追溯到小马快递的日子。

为了避免任何误解，值得注意的是没有多路访问协议能保证可靠传送。即使没有发生冲突，也有这样或者那样的原因使得接收器错误地复制了帧的某些部分。因此，要由链路层的其他部分或比链路层更高的层次来提供数据传输的可靠性。

4.2 多路访问协议

分配一个多路访问信道的算法有许多。在下面这一节中，我们将学习一些比较有意思的算法，并给出在实际中如何应用它们的实例。

4.2.1 ALOHA

我们的第一个 MAC 故事开始于 20 世纪 70 年代“原始的”夏威夷。在这种情况下，“原始的”（pristine）一词可以解释为“没有一个可运行的电话系统”。这并没有使生活在这里的夏威夷大学研究员 Norman Abramson 和他的同事感觉愉快，他们正试图把偏远岛屿上的用户连接到檀香山的主计算机。把自己的电缆穿过太平洋海底显然不是个办法，因此他们寻找着不同的解决方案。

他们找到了一种用于短程无线电通信的方法。所有的用户终端共享同一个上行频率给中央计算机发送帧。其中包括了一个简单而巧妙的方法来解决信道分配问题。自此以后，他们的工作得到了许多研究者的扩充（Schwartz 和 Abramson, 2009）。虽然 Abramson 的工作（称为 ALOHA 系统）使用了地面无线电广播，其基本思路同样适用于任何非协调用户竞争使用单个共享信道的系统。

我们在这里将要讨论两个版本的 ALOHA：纯 ALOHA 和分槽 ALOHA。它们的区别在于时间是连续的，那就是纯粹版的 ALOHA；或者时间分成离散槽，所有帧都必须同步到

时间槽中。

纯 ALOHA

ALOHA 系统的基本思想非常简单：当用户有数据需要发送时就传输。当然，这样可能会产生冲突，冲突的帧将被损坏。发送方需要某种途径来发现是否发生了冲突。在 ALOHA 系统中，每个站在给中央计算机发送帧之后，该计算机把该帧重新广播给所有站。因此，那个发送站可以侦听来自集线器的广播，以此确定它的帧是否发送成功。在其他系统中，比如有线局域网中，发送方在发送的同时能侦听到冲突的发生。

如果帧被损坏了，则发送方要等待一段随机时间，然后再次发送该帧。等待的时间必须是随机的，否则同样的帧会一次又一次地冲突，因为冲突帧被重发的节奏完全一致。如果系统中多个用户共享同一个信道的方法会导致冲突，则这样的系统称为竞争（contention）系统。

图 4-1 给出了一个 ALOHA 系统中帧的框架结构。我们使所有的帧具有同样的长度，因为对于 ALOHA 系统，采用统一长度的帧比长度可变的帧更能达到最大的吞吐量。

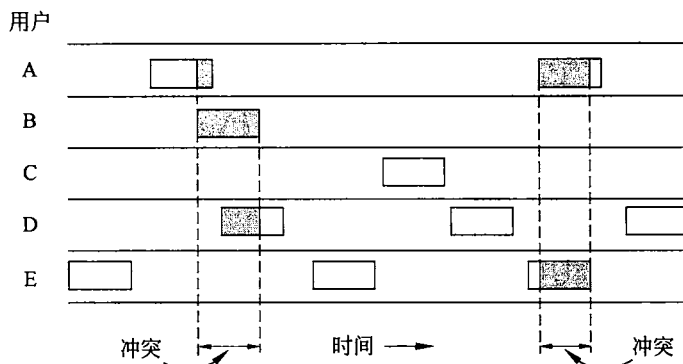


图 4-1 在纯 ALOHA 中帧的发送次序是任意的

无论何时，只要两个帧在相同时间试图占用信道，冲突就会发生（如图 4-1 所示），并且两帧都会被破坏。如果新帧的第一位与几乎快传完的前一帧的最后一位重叠，则这两帧都将被彻底毁坏（即具有不正确的校验和），稍后都必须被重传。校验和不可能（也不应该）区分出是完全损坏还是局部差错。坏了就是坏了。

一个有趣的问题是：ALOHA 信道的效率怎么样？换句话说，在这样混乱的情况下，能够侥幸逃脱冲突而被传输出去的帧占多大比例？我们首先考虑这样的情形：有无穷多个交互用户坐在他们的终端（站）前面。每个用户总是处于两种状态中的一种：敲键或等待。刚开始时，所有的用户都处于敲键的状态。当输入完一行之后，用户停止敲键，开始等待应答；然后站在共享信道上给中央计算机发送一个包含了该行字符的帧，并且检查信道看是否传输成功。如果传输成功，则用户会看到应答，回到敲键状态继续输入。如果不成功，则在站一次次重传该帧时用户继续等待，直到该帧被成功发送出去为止。

我们用“帧时”（frame time）来表示传输一个标准的、固定长度的帧所需要的时间（即帧的长度除以比特率）。现在我们假定站产生的新帧可以模型化为一个平均每“帧时”产生 N 个帧的泊松分布（假设存在无穷多个用户是必要的，因为这样可以确保 N 不会随着用户变成阻塞状态而下降）。如果 $N > 1$ ，则用户群生成帧的速率大于信道的处理速度，因此，

几乎每个帧都要经受冲突。为了取得合理的吞吐量，我们应该期望 $0 < N < 1$ 。

除了新生成的帧以外，每个站还会产生由于先前遭受冲突而重传的那些帧。我们进一步假设在每个“帧时”中，老帧和新帧合起来也符合泊松分布，每帧时的平均帧数为 G 。显然， $G \geq N$ 。在负载较低的情况下（即 $N < 0$ ），冲突很少发生，因此重传也很少，于是 $G < N$ 。在负载较高时，将会有很多冲突，所以 $G > N$ 。在所有这些负载的情况下，吞吐量 S 就是负载 G 乘以成功传输的概率 P_0 —— $S = GP_0$ ，其中 P_0 是一帧没有遭受冲突的概率。

如果从一帧被发送出去开始算起，在一个“帧时”内没有发出其他的帧，则这一帧不会遭到冲突，如图 4-2 所示。在什么样的条件下，图中的阴影帧将毫无损坏地到达目的地？假设发送一帧所需的时间为 t 。如果其他用户在 $t_0 \sim t_0 + t$ 之间生成了一帧，则该帧的结尾部分将与阴影帧的开始部分发生冲突。实际上，阴影帧的命运在它的第一位被发出去之前就已经注定了。但是，由于在纯 ALOHA 中，站在传输之前并不去侦听信道，所以，它无法知道是否有其他的帧已经在使用信道上了。类似地，在 $t_0 + t \sim t_0 + 2t$ 之间开始发送的任何其他帧都会和阴影帧的结尾部分冲突。

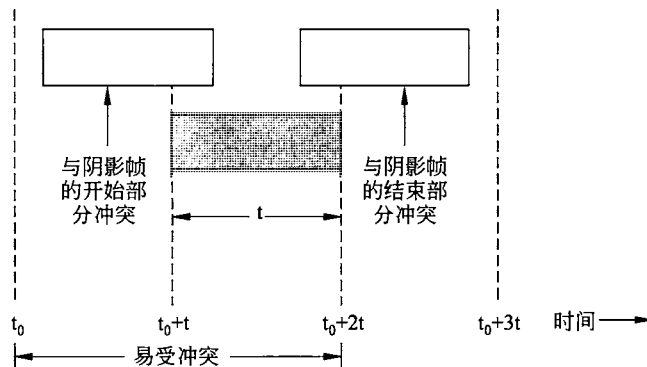


图 4-2 阴影帧的易受冲突周期

在给定的一个“帧时”内希望有 G 帧，但生成 k 帧的概率服从泊松分布：

$$\Pr[k] = \frac{G^k e^{-G}}{k!} \quad (4-2)$$

所以，生成零帧的概率为 e^{-G} 。在两个“帧时”长的间隔中，生成帧的平均数是 $2G$ 。因此，在整个易受冲突期中，不发送帧的概率是 $P_0 = e^{-2G}$ 。利用 $S = GP_0$ ，则可以得到：

$$S = Ge^{-2G}$$

流量与吞吐量之间的关系如图 4-3 所示。最大的吞吐量出现在当 $G = 0.5$ 时， $S = 1/2e$ ，大约等于 0.184。换句话说，我们可以希望的最好信道利用率为 18%。这个结果并不令人鼓舞，但是对于这种任何人都可以随意发送的传输方式，要想达到 100% 的成功率几乎是不可能的。

分槽 ALOHA

ALOHA 出现不久，Roberts 发表了一种能将 ALOHA 系统的容量增加一倍的方法 (Roberts, 1972)。他的建议是将时间分成离散的间隔，这种时间间隔称为时间槽 (slot)，每个时间槽对应于一帧。这种方法要求用户遵守统一的时间槽边界。取得同步时间的一种办法是由一个特殊的站在每个间隔起始时发出一个脉冲信号，就好像一个时钟一样。

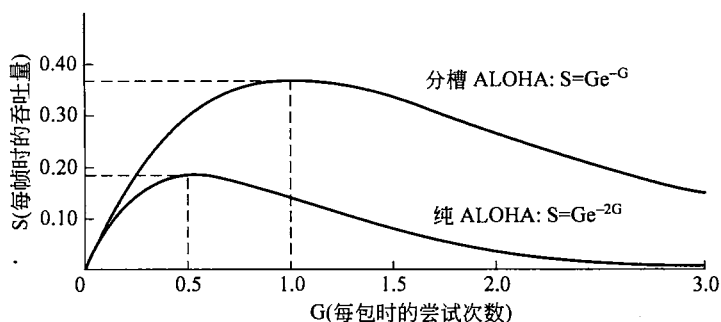


图 4-3 ALOHA 系统的吞吐量与负载的关系

Roberts 方法称为分槽 ALOHA (slotted ALOHA)。与纯 ALOHA 不同的是, 在分槽 ALOHA 中, 站不允许用户每次敲入回车键就立即发送帧。相反, 它必须要等到下一个时槽的开始时刻。因此, 连续纯 ALOHA 变成了离散的 ALOHA, 这将易受冲突期减小了一半。为了解清楚这点, 请看图 4-3, 并且想象现在可能发生的冲突。在测试帧所在的同一个时间槽中没有其他流量的概率是 e^{-G} , 于是可以得到:

$$S = Ge^{-G} \quad (4-3)$$

正如你可以从图 4-3 中看到的那样, 分槽 ALOHA 的尖峰在 $G=1$ 处, 此时吞吐量为 $S=1/e$, 大约等于 0.368, 是纯 ALOHA 的两倍。如果系统运行在 $G=1$ 处, 则空时间槽的概率为 0.368 (从等式 (4-2) 可以得出)。使用分槽 ALOHA, 我们期望的最好结果是 37% 为空时间槽、37% 为成功, 剩下 26% 为冲突。如果在更高的 G 值上运行, 则空时间槽数会降低, 但冲突时间槽数会呈指数增长。为了看出冲突时间槽数是如何随着 G 的变化而快速增长, 请考虑一个测试帧的传输过程。该测试帧能够避免一次冲突的概率是 e^{-G} , 即所有其他用户在该时间槽中静止不发帧的概率。于是, 冲突的概率为 $1-e^{-G}$ 。需要 k 次尝试才能成功传输的概率 (即 $k-1$ 次冲突之后才有一次成功的概率) 为:

$$P_k = e^{-G}(1-e^{-G})^{k-1}$$

于是, 每帧传输次数期望 E , 即终端键入一行的概率为:

$$E = \sum_{k=1}^{\infty} kP_k = \sum_{k=1}^{\infty} ke^{-G}(1-e^{-G})^{k-1} = e^{-G}$$

所以, E 随 G 呈指数增长的结果是信道负载的微小增长也会极大地降低信道的性能。

分槽 ALOHA 引起关注的一个原因在于它的重要性在刚开始时没有得到重视。它是在 20 世纪 70 年代被设计出来, 曾经用在一些实验系统中, 之后差不多就被大家遗忘了。当通过有线电视电缆访问 Internet 技术被发明出来时, 立即出现了一个问题, 那就是如何在多个竞争用户之间分配一条共享信道, 于是分槽 ALOHA 被人们从遗忘的角落中翻了出来, 从而拯救了世界。后来, 多个 RFID 标签和同一个 RFID 读写器通信时也出现了同样的问题, 只是表现形式不同。又是分槽 ALOHA, 和其他想法结合起来再次成了救世主。经常会出现这样的情况: 一些非常完善有效的协议由于政策上的原因 (比如某个大公司希望每个人都按照它的方式来行事), 或者因为不断变化的技术发展趋势而被弃置不用。然后, 多年以后, 一些聪明人就会发现某个长期被束之高阁的协议恰好可以解决他们的当前问题。出于这样的原因, 在本章我们将学习一些非常优秀的协议, 尽管它们目前尚未得到广泛应

用，只要有足够的网络设计师了解它们，在将来的应用中它们很有可能会发挥作用。当然，我们也会学习许多当前正在使用的协议。

4.2.2 载波侦听多路访问协议

利用分槽 ALOHA，可以达到的最佳信道利用率是 $1/e$ 。这么低的结果并不令人惊奇，因为每个站都可以随意地发送数据，它并不知道其他站是否也在发送数据。所以，频繁地发生冲突是难免的。然而，在局域网中，站是完全有可能检测到其他站当前在做什么，然后再根据情况调整自己的行为。这些网络可以获得比 $1/e$ 好得多的利用率。在这一小节中，我们将讨论一些提高性能的协议。

如果在一个协议中，站监听是否存在载波（即是否有传输），并据此采取相应的动作，则这样的协议称为载波侦听协议（carrier sense protocol）。很久以前许多这类协议就被提了出来，而且都被详细地分析过了。例如，参考（Kleinrock 和 Tobagi, 1975）。下面我们看几个载波侦听协议。

坚持和非坚持 CSMA

我们将要学习的第一个载波侦听协议称为 1-坚持载波检测多路访问（CSMA, Carrier Sense Multiple Access）。这是最简单的 CSMA 方案。当一个站有数据要发送时，它首先侦听信道，确定当时是否有其他站正在传输数据；如果信道空闲，它就发送数据。否则，如果信道忙，该站等待直至信道变成空闲；然后，站发送一帧。如果发生冲突，该站等待一段随机的时间，然后再从头开始上述过程。这样的协议之所以称为 1-坚持，是因为当站发现信道空闲时，它传输数据的概率为 1。

除了罕见的多个站同时发送情况外，你或许期望这个方案能避免冲突，但实际上它不能。如果两个站在第三个站的发送过程中准备好了拟发送的数据，它们俩都会礼貌地等待，直到当前的传输结束；然后双方将确定同时开始传输，这种行为显然会导致冲突发生。如果它们不那么急躁，冲突的可能性将会少得多。

更微妙的是，信号的传播延迟对冲突有着重大影响。这里存在一个时机，在某个站开始发送后，另一个站也刚好做好发送的准备并侦听信道。如果第一个站的信号没能到达第二个站，后者侦听到信道是空闲的，因而也开始发送，由此导致双方的冲突。可见这个机会取决于信道上适合的帧数，或信道的带宽延迟积（bandwidth-delay product）。如果信道只够容纳帧的很小一部分（这种情况符合大多数局域网的环境），由于信号的传播延迟小，冲突发生的机会就小。带宽延迟积越大，这种影响就变得愈加重要，因而协议的性能越差。

即便如此，上述协议的性能也比纯 ALOHA 协议要好得多，因为这两个站都非常礼貌，不会去打扰第三个站的帧。确切地，同样的结论也适用于分槽 ALOHA。

第二个载波侦听协议是非坚持 CSMA（nonpersistent CSMA）。在这个协议中，站在试图发送数据之前要理智得多，不像前一个协议那样贪婪。和前一个协议一样，站在发送数据之前要先侦听信道。如果没有其他站在发送数据，则该站自己开始发送数据。然而，如果信道当前正在使用中，则该站并不持续对信道进行监听，以便传输结束后立即抓住机会发送数据。相反，它会等待一段随机时间，然后重复上述算法。因此，该算法将会导致更

好的信道利用率,但是比起 1-坚持 CSMA,也带来了更大的延迟。

最后一个协议是 p -坚持 CSMA (p -persistent CSMA)。它适用于分时间槽的信道,其工作方式如下所述。当一个站准备好要发送的数据时,它就侦听信道。如果信道是空闲的,则它按照概率 p 发送数据;而以概率 $q=1-p$,将此次发送推迟到下一个时间槽。如果下一个时间槽信道也是空闲的,则它还是以概率 p 发送数据,或者以概率 q 再次推迟发送。这个过程一直重复,直到帧被发送出去,或者另一个站开始发送数据。如果发生了后一种情况,那么这个极不走运的站按照冲突发生时的处理过程一样(即等待一段随机的时间,然后再重新开始)。如果该站刚开始时就侦听到信道为忙,则它等到下一个时间槽,然后再应用上面的算法。IEEE 802.11 对 p 坚持 CSMA 作了细微的改良,我们将在 4.4 节讨论它。

图 4-4 显示了上述 3 个协议以及纯 ALOHA 和分槽 ALOHA 的可计算吞吐量和负载之间的关系。

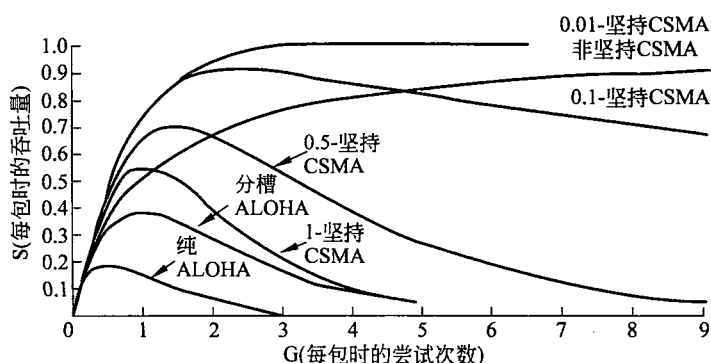


图 4-4 不同随机访问协议的信道使用率与负载比较

带冲突检测的 CSMA

坚持和非坚持 CSMA 协议无疑是对 ALOHA 的改进,因为这些协议都确保了信道忙时,所有的站都不再传送数据。然而,如果两个站侦听到信道为空,并且同时开始传输,则它们的信号仍然会产生冲突。因此,另一个改进是每个站快速检测到发生冲突后立即停止传输帧(而不是继续完成传输),因为这些帧已经无可挽回地成为乱码。这种策略可以节省时间和带宽。

这种协议称为带冲突检测的 CSMA (CSMA/CD, CSMA with Collision Detection)。它是经典以太网的基础,所以,值得我们专门花一点时间来详细地介绍它。重要的是要认识到,冲突检测是一个模拟过程。站的硬件在传输时必须侦听信道。如果它读回的信号不同于它放到信道上的信号,则它就知道发生了碰撞。言外之意是接收信号相比发射信号不能太微弱(这对无线来说很难做到,因为接收信号可能 1 000 000 倍弱于发射信号),并且必须选择能被检测到冲突的调制解调技术(比如,两个 0 伏信号的冲突很可能无法检测到)。

如同许多其他 LAN 协议一样,CSMA/CD 也使用了图 4-5 所示的概念模型。在标记为 t_0 点,一个站已经完成了帧的传送,其他需要发送帧的站现在可以试图发送了。如果有两个或者多个站同时进行传送,冲突就会发生。如果一个站检测到冲突,它立即中止自己的传送,等待一段随机时间,然后再重新尝试传送(假定在此期间没有其他站已经开始传送)。



因此，我们的 CSMA/CD 模型将由交替出现的竞争期、传输期，以及当所有站都静止的空闲期（比如没有传输任务）组成。

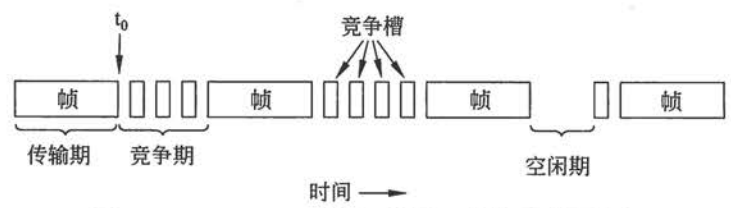


图 4-5 CSMA/CD 可能处于竞争、传输或空闲状态

现在我们来查看竞争算法的细节。假定两个站同时在 t_0 时刻开始传送数据。它们需要多长时间才能意识到发生了冲突呢？该问题的答案对于确定竞争周期的长度至关重要，进而会影响到延迟和吞吐量。

检测冲突的最小时间恰好是将信号从一个站传播到另一个站所需要的时间。基于这个信息，你可能会认为一个站在开始发送后，经过一段特定的时间还未监听到冲突，它就可以确定自己“抓住”（seize）了电缆。这里“抓住”的意思是指所有其他站都知道该站正在传送数据，所以不会干扰它；而那段特定的时间表示整条线缆的传播时间。这个结论是错误的。

考虑下面给出的最坏情形。假设两个相距最远的站传播信号所需要的时间为 τ 。在 t_0 时刻，一个站开始传送数据。在 $t_0 + \tau - \epsilon$ 时刻，也就是在信号到达最远那个站之前的一刹那，那个站也开始传输。当然，它几乎立刻就检测到冲突，并且立即停止了传输，但由这次冲突引起的微小噪声尖峰要到 $2\tau - \epsilon$ 时刻才能回到原来的那个发送站。换句话说，在最差的情况下，只有当一个站传输了 2τ 之后还没有监听到冲突，它才可以确保自己经抓住了信道。

有了这种认识，我们可以把 CSMA/CD 竞争看成是一个分槽 ALOHA 系统，时间槽宽度为 2τ 。在 1 千米长的同轴电缆上， $\tau < 5$ 微秒。CSMA/CD 和分槽 ALOHA 的区别在于，只有一个站能用来传输的时间槽（即信道被抓住了）后面紧跟的那些时间槽被用来传输该帧的其余部分。如果帧时相比传播时间长很多，这种差异将能大大提高协议的性能。

4.2.3 无冲突协议

在 CSMA/CD 中，一旦站已经确定无疑地抓住了信道，冲突就不会再发生；尽管如此，在竞争期中冲突仍有可能发生。这些冲突严重地影响了系统的性能，特别是当带宽延迟积很大，比如电缆很长（即 τ 很大）而帧的长度又很短时。冲突不仅降低了带宽，而且使得发送一个帧的时间变得动荡不定，这样就无法很好地适应实时流量，比如 IP 语音。而且 CSMA/CD 也不是普遍适用的。

在本节，我们将介绍一些协议，它们以根本不可能产生冲突的方式解决了信道竞争问题，即使在竞争期中也不会发生冲突。大多数这样的协议目前并没有被用在主流系统中；但是，在一个快速变化的领域，拥有一些具有优异特性可适用于未来系统的协议，通常是一件好事。

在接下来描述的协议中，我们假定共有 N 个站，每个站都有唯一的地址，地址范围从

0 到 $N-1$ 。有些站在一部分时间中可能是不活跃的，不过这无关紧要。我们还假定传播延迟可以忽略不计。但是，基本问题仍然存在：在一次成功的传输之后哪个站将获得信道？我们继续使用带有离散竞争时间槽的图 4-5 模型。

位图协议

我们要介绍的第一个无冲突协议采用了基本位图法 (basic bitmap method)，每个竞争期正好包含 N 个槽。如果 0 号站有一帧数据要发送，则它在第 0 个槽中传送 1 位。在这个槽中，不允许其他站发送。不管 0 号站做了什么，1 号站有机会在 1 号槽中传送 1 位，但是只有当它有帧在排队等待时才这样做。一般地， j 号站通过在 j 号槽中插入 1 位来声明自己有帧要发送。当所有 N 个槽都经过后，每个站都知道了哪些站希望传送数据。这时候，它们便按照数字顺序开始传送数据了，如图 4-6 所示。

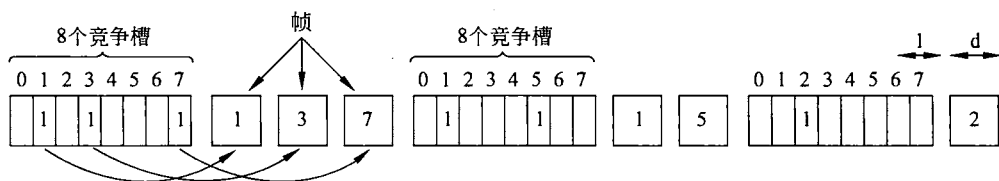


图 4-6 基本位图协议

由于每个站都同意下一个是谁传输，所以永远也不会发生冲突。当最后一个就绪站传完它的帧后，这是所有站都很容易检测到的事件，于是，另一个 N 位竞争期又开始了。如果一个站在它对应的位槽刚刚经过就准备好了要发送的数据，那它就非常不幸，只能保持沉默；直到每个站都获得了发送数据的机会，新的位图再次到来，它才能通过位槽表明自己有传输的意愿。

像这样在实际传输数据之前先广播自己有发送数据愿望的协议，称为预留协议 (reservation protocol)。现在我们简要分析这个协议的性能。为了简便起见，我们将用竞争槽作为单位来计量时间，假定数据帧由 d 个时间单位构成。

在负载很低的情况下，数据帧非常少，位图简单地一次又一次地重复出现。我们从序号较低的站，比如 0 号站或者 1 号站的角度来考虑。典型情况下，当它已经做好发送数据的准备时，“当前”槽将处于位图中间的某个地方。平均而言，该站必须等待完成当前扫描的 $N/2$ 个槽，再等待完成下一次扫描的另外 N 个槽，然后才能开始传输数据。

从高序号的站来看则情形会好很多。一般地，它只需等待半个扫描周期 ($N/2$ 个位槽) 就可以开始传输数据了。高序号的站往往不必等待到下一次扫描。由于低序号的站必须等待平均 $1.5N$ 个槽，而高序号的站必须等待平均 $0.5N$ 个槽，因此对所有站而言，要平均等待 N 个槽。

在低负载情况下的信道效率很容易计算。每一帧的额外开销是 N 位，数据长度为 d 位，于是信道利用率为 $d/(N+d)$ 。

在高负载的情况下，若所有站在任何时候都有数据要发送，则 N 位竞争期被分摊到 N 个帧上，因此，每一帧的额外开销只有 1 位，或者，信道利用率为 $d/(d+1)$ 。一帧的平均延迟等于它在站内的排队时间，加上它到达队列头部之后另外的 $(N-1)d+N$ 时间。它要等待的这个间隔是所有其他站轮流发送一帧以及一个位图的时间。

令牌传递

位图协议的实质是让每个站以预定义的顺序轮流发送一帧。完成同样事情的另一种方法是通过传递一个称为令牌（token）的短消息，该令牌同样也是以预定义的顺序从一个站传到下一个站。令牌代表了发送权限。如果站有个等待传输的帧队列，当它接收到令牌就可以发送帧，然后再把令牌传递到下一站。如果它没有排队的帧要传，则它只是简单地把令牌传递下去。

在令牌环（token ring）协议中，网络的拓扑结构被用来定义站的发送顺序。所有站连接成一个单环结构，一个站依次连接到下一个站。因此令牌传递到下一站只是单纯地从一个方向上接收令牌和在另一个方向上发送令牌，如图 4-7 所示。帧也按令牌方向传输。这样，它们将绕着环循环，到达任何一个目标站。然而，为了阻止帧陷入无限循环（像令牌一样），一些站必须将它们从环上取下来。这个站或许是最初发送帧的原始站，在帧经历了一个完整的环游后将它取下来，或者是帧的指定接收站。

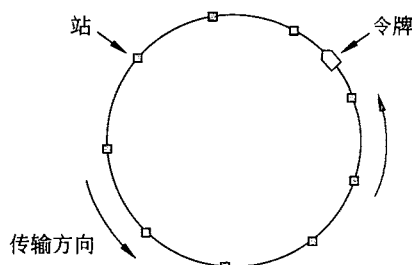


图 4-7 令牌环

请注意，我们并不需要一个物理环来实现令牌传递。相反，连接各站的信道可以是一根长总线。每个站通过该总线按照预定义的顺序把令牌发给下一站。令牌的拥有者可以利用总线发送帧，如前所述。这个协议称为令牌总线（token bus）。

令牌传递的性能类似于位图协议，尽管现在竞争槽和帧被混合在一个周期中。发送一帧后，每个站必须等待所有 N 个站（包括其自身）把令牌发给各自的邻居，以及其他 $N-1$ 个站发送完一帧（如果它们有帧需要发送）。两者的一个细微差别在于，因为在周期中所有的位置是均等的，所以不存在偏向低编号或者高编号一说。对于令牌环，每个站在协议采取下一步动作之前只将令牌尽可能发送给它的邻居。在协议前进到下一步之前不需要将每个令牌传播给全部站。

令牌环随之显身于 MAC 协议，这些协议具有某种一致性。早期的令牌环协议（称为令牌环（Token Ring），并标准化为 IEEE 802.5）在 20 世纪 80 年代非常流行，是经典以太网之外的另一种局域网选择。到了 20 世纪 90 年代，一种更快的令牌环，称为光纤分布式数据接口（FDDI, Fiber Distributed Data Interface）被交换式以太网击败。2000 年后，一个称为弹性数据包环（RPR, Resilient Packet Ring）的令牌环被定义为 IEEE 802.17，这是对 ISP 使用的城域网组合所制定的标准化。我们很想知道 2010 年以后将提供什么样的服务。

二进制倒数计数

基本位图协议存在一个问题。每个站的开销是 1 位，所以该协议不可能很好地扩展到含有上千个站的网络中，扩展后的令牌传递也有同样的问题。通过使用二进制的站地址，我们可以做得更好。如果一个站想要使用信道，它就以二进制位串的形式广播自己的地址，从高序的位开始。假定所有地址都有同样的长度。不同站地址中相同位在同时发送时被信道布尔或（BOOLEAN OR）在一起。我们把这样的协议称为二进制倒数计数（binary countdown）协议，它曾经被用在 Datakit 中（Fraser, 1987）。它隐式地假设传输延迟可忽

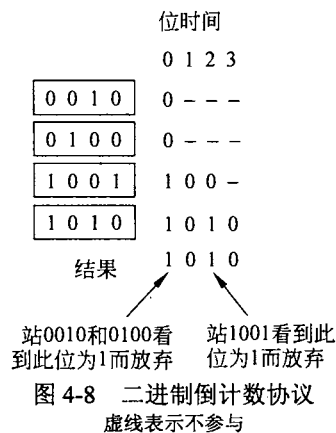
略不计, 因此, 所有站几乎能同时看到地址宣告位。

为了避免冲突, 必须使用一条仲裁规则: 一个站只要看到自己的地址位中的 0 值位置被改写成了 1, 则它必须放弃竞争。例如, 如果站 0010、0100、1001 和 1010 都试图要获得信道, 在第一位时间中, 这些站分别传送 0、0、1 和 1。它们被 OR 在一起, 得到 1。站 0010 和 0100 看到了 1, 它们立即明白有高阶的站也在竞争信道, 所以它们放弃这一轮的竞争。而站 1001 和 1010 则继续竞争信道。

接下来的位为 0, 于是两者继续竞争; 再接下来的位为 1, 所以站 1001 放弃。最后的胜者是 1010, 因为它有最高的地址。在赢得了竞争后, 它现在可以传输一帧, 之后又开始新一轮竞争。该协议由图 4-8 举例说明。该协议具有这样一种特性, 高阶站的优先级比低序站的优先级高, 这可能是好事, 也可能是坏事, 取决于上下文。

这种方法的信道利用率为 $d/(d+\log_2 N)$ 。然而, 如果精心设计帧格式, 使得发送方的地址正好是帧内的第一个字段, 那么, 甚至 $\log_2 N$ 位也不会被浪费, 所以信道利用率为 100%。

二进制倒数计数协议是一个简单的、精致的和高效的协议实例, 它有待于被重新发现。希望有一天它能找到一个新的用武之地。



4.2.4 有限竞争协议

如何在一个广播网络中获取信道, 我们已经考虑了两种基本策略: 一种是竞争的方法, 如同 CSMA 的做法那样; 另一种是无竞争协议。每一种都可以用两个重要性能指标来衡量: 低负载下的延迟, 以及高负载下的信道利用率。在负载较轻的情况下, 竞争方法 (即纯 ALOHA 或者分槽 ALOHA) 更为理想, 因为它的延迟较短 (冲突很少发生)。随着负载的增加, 竞争方法变得越来越缺乏吸引力, 因为信道仲裁所需要的开销变得越来越大。而对于无冲突协议, 则结论刚好相反。在低负载情况下, 它们有相对高的延迟, 但是随着负载的增加, 信道的效率反而得到提高 (因为开销是固定的)。

显然, 如果我们能够把竞争协议和无冲突协议的优势结合起来, 那就太好了。这样得到的新协议在低负载下采用竞争的做法而提供较短的延迟, 但在高负载下采用无冲突技术, 从而获得良好的信道效率。我们把这样的协议称为有限竞争协议 (limited-contention protocol), 实际上这样的协议的确存在, 我们将用它来结束关于载波侦听网络的学习。

到现在为止我们学习过的竞争协议都是对称的, 也就是说, 每个站企图获得信道的概率为 p , 并且所有的站都使用同样的 p 。很有意思的是, 如果协议为不同的站分配不同的概率, 有时系统的整体性能会有所提高。

在讨论非对称协议之前, 让我们快速回顾一下对称协议的性能情况。假设共有 k 个站竞争信道的使用权。每个站在每个时间槽中的传输概率为 p 。那么, 在一个给定的时间槽中, 某个站能够成功地获得信道的概率是任何一个站以概率 p 传输, 而所有 $k-1$ 个站以 $1-p$

概率把传输延缓到下一个时间槽，这个值为 $kp(1-p)^{k-1}$ 。为了找到 p 的最优值，我们按照 p 对它求微分，再将结果设置为 0，解出 p 值。这样做之后，我们发现， p 的最佳值为 $1/k$ 。将 $p=1/k$ 代入，则得到

$$\text{Pr}[p \text{ 为最佳值时的成功概率}] \left(\frac{k-1}{k} \right)^{k-1} \quad (4-4)$$

这个概率的曲线如图 4-9 所示。对于站数较少的情形，成功的概率很高；但是，一旦站的数量达到 5 个以后，概率很快下降，接近于它的逼近值 $1/e$ 。

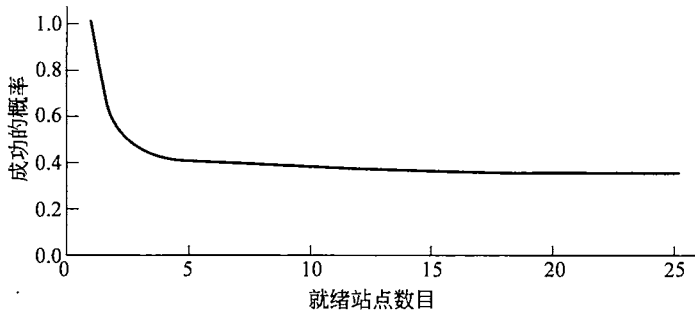


图 4-9 对称竞争信道的获得概率

从图 4-9 可以很明显地看到，只要减少参与竞争的站数量，则站获得信道的概率就会增加。有限竞争协议正是这样做的。它们首先将所有的站划分成组（这些组不必是两两不相交的）。只有 0 号组的成员才允许竞争 0 号时间槽。如果该组中的一个成员竞争成功了，则它获得信道，可以传送它的帧。如果该时间槽是空闲的，或者发生了冲突，则 1 号组的成员竞争 1 号时间槽，以此类推。通过适当的分组办法，可以减少每个时间槽中的竞争数量，从而使得每个时间槽中的行为接近图 4-9 的左侧。

协议的诀窍在于如何将站分配到各个时间槽中。在讨论一般情形以前，我们先考虑一些特殊情形。在一种极端情况下，每个组只包含一个站。这样的分配方案可以保证永远不会发生冲突，因为对于任何给定的时间槽，至多只有一个站参与竞争。前面我们已经看到过这样的协议（比如二进制倒数计数协议）。另一种特殊的情形是每个组有两个站。在一个时间槽中，两个站都要传送数据的概率是 p^2 ，对于很小的 p ，这个值可以忽略不计。随着分配在同一个时间槽中的站数越来越多，冲突的概率也随之增加，但是给予各站竞争机会所需的位图扫描长度却缩小了。有限的情况是每个组包含全部的站（即分槽 ALOHA）。我们所需要的是一种动态地将站分配到时间槽的方法，当负载很低时，每个时间槽中的站点数量就多一些；当负载很高时，每个时间槽中的站点数量就少一些，甚至只有一个站。

自适应树遍历协议

有一种特别简单方法可以用来执行必要的信道分配任务，那就是采用第二次世界大战中美国军方为了测试士兵是否感染梅毒而设计的算法（Dorfman, 1943）。简短来说，军方从 N 个士兵身上提取血样。然后从每份血样中各取一部分倒入同一个试管中，再对这份混合的血样进行抗体测试。如果没有发现抗体，则所有的士兵都是健康的。如果出现了抗体，则再准备两份新的混合血样，一份由 1 到 $N/2$ 士兵的血样混合而成，另一份由剩余士兵的血样混合而成。这个过程重复递归，直到找出被感染的士兵。

至于该算法的计算版本 (Capetanakis, 1979) 很自然地把站看作是二叉树的叶节点, 如图 4-10 所示。在一次成功传送之后的第一个竞争槽, 即 0 号槽中, 允许所有的站尝试获取信道。如果它们之中的某一个获得了信道, 则很好; 如果发生了冲突, 则在 1 号槽中, 只有位于树中 2 号节点之下的那些站才可以参与竞争。如果其中的某个站获得了信道, 则在该站发送完一帧之后的那个槽被保留给位于节点 3 下面的那些站。另一方面, 如果节点 2 下面的两个或者多个站都要传输数据, 则在 1 号槽中就会发生冲突, 此时, 下一个槽, 即 2 号槽就由位于节点 4 下面的站来竞争。

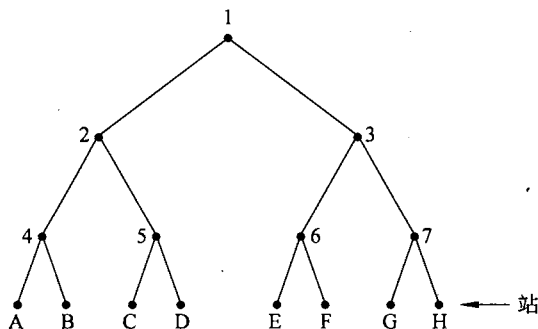


图 4-10 包含 8 个站的树

本质上, 如果在 0 号槽中发生了冲突, 则整棵树都会被遍历到, 按深度优先策略找到所有就绪站。每一个位槽都跟树中某些特定的节点相关联。如果发生了冲突, 则在该节点的左子节点和右子节点上继续递归地进行搜索。如果一个位槽是空闲的, 或者在位槽中只有一个站传送数据, 则停止该节点的搜索, 因为这表明已经找到了该节点下面所有就绪的站 (如果有多个就绪站, 一定会发生冲突)。

当系统负载较重, 将 0 号槽专门指定给节点 1 几乎不值得, 因为只有当精确地只有一个站有帧要发送, 这才是有意义; 而当负载较重时这种事件发生的可能性非常小。类似地, 有人可能会说, 基于同样的理由, 节点 2 和节点 3 也可以跳过去。考虑更为一般的情形, 到底应该从树的哪一级开始搜索呢? 很明显, 系统负载越重, 则越是应该从树的下面节点开始搜索。我们假设每个站对全部就绪站的数目有良好的估算 q , 例如, 可以从监测当前的流量来推导出就绪站的估算数。

若要继续, 让我们对树的级数从上往下进行编号, 在图 4-10 中, 节点 1 位于第 0 级, 节点 2 和节点 3 位于第 1 级, 以此类推。请注意, 第 i 级上的每个节点是其下面总站数的 2^{-i} 。如果 q 个就绪站均匀分布, 则在第 i 级上某一个特定节点下面期望的就绪站数是 $2^{-i}q$ 。直观上, 我们期望开始搜索的最优级数应该是每个槽中参与竞争的平均站数为 1, 也就是说, 在这级上, $2^{-i}q=1$ 。求解这个方程, 我们可以得到 $i=\log_2 q$ 。

这个基本算法已经有了大量的改进算法, (Bertsekas 和 Gallager, 1992) 对这些算法作了详细的讨论。例如, 考虑这样一种情况: 只有站 G 和 H 想要发送数据。在结点 1 上, 发生冲突, 所以节点 2 下面的站开始竞争, 却发现它是空闲的; 此时探测节点 3 毫无意义, 因为它肯定会冲突 (我们已经知道在节点 1 下面有两个或者多个站就绪, 并且这些站都不在节点 2 的下面, 所以, 它们肯定都在节点 3 的下面)。对节点 3 的探测可以跳过去, 直接探测节点 6。当这次探测结果仍然是空时, 节点 7 可以跳过去, 尝试节点 G 了。

4.2.5 无线局域网协议

笔记本电脑通过无线电进行通信，它们组成的系统可以看作是无线局域网（wireless LAN），就像我们在 1.5.3 节讨论的一样。这样的局域网是广播信道的一个例子。它具有某些和有线局域网不同的属性，这些属性导致了无线局域网不同的 MAC 协议。在本节，我们将研究这些协议中的一些。在 4.4 节，我们将着眼于 802.11（WiFi）细节。

无线局域网的一种常见配置是在一座办公大楼内，有策略地放置一些环绕大楼的接入点（AP）。AP 通过铜缆或光纤连接在一起，并为与之联系的站提供接入服务。如果 AP 和笔记本电脑的发射功率调整在一个数十米的范围内，附近的房间就变得像单个蜂窝，而整个大楼就像我们在第 2 章学习的蜂窝电话系统；但有一点除外，那就是每个蜂窝只有一个信道可用。这个信道被蜂窝内所有的站共享，包括 AP。它通常提供 Mbps 的带宽，最大可高达 600 Mbps。

我们已经说过，无线通信系统通常不能检测出正在发生的冲突。站接收到的信号可能很微弱，也许比它发出去的信号弱上百万倍。要发现这样的冲突信号就像在海上寻找涟漪一样的困难。相反，确认机制可用在事后发现冲突和其他错误。

无线局域网和有线局域网之间还存在着一个更重要的差异。由于无线电传输范围有限，无线局域网中的站或许无法给所有其他站发送帧，也无法接收来自所有其他站的帧。在有线局域网中，一个站发出一帧，所有其他站都能接收到。正是由于无线局域网缺乏这种属性，导致了它的一系列复杂性。

我们将做出简化的假设：即每个无线电发射器有某个固定的传播范围，用一个圆形覆盖区域表示，在这区域内的另一个站可以侦听并接收该站的传输。重要的是要认识到，实际上的覆盖区域没有那么规则，因为无线电信号的传播依赖于所在的环境。墙壁和其他障碍物对信号都会造成衰减和反射，这些可能会导致信号在不同方向上表现出显著不同。但是一个简单的圆形模式有助于达到我们的描述目的。

使用无线 LAN 的一种单纯想法是尝试使用 CSMA：每个站侦听是否有其他站在传输，并且只有当没有其他站在传送数据时它才传输。这种方法的麻烦在于协议并未考虑无线传输特性，因为这里的冲突发生在接收方，而不是发送方。为了看清楚问题的实质，考虑如图 4-11 所示的 4 个无线站的情形。对于我们的问题描述来说，这些无线站到底是基站还是笔记本电脑无关紧要。无线电的覆盖范围是这样的：A 和 B 都在对方的范围内，可能潜在地相互之间有干扰；C 也可能潜在地干扰到 B 和 D，但不会干扰 A。

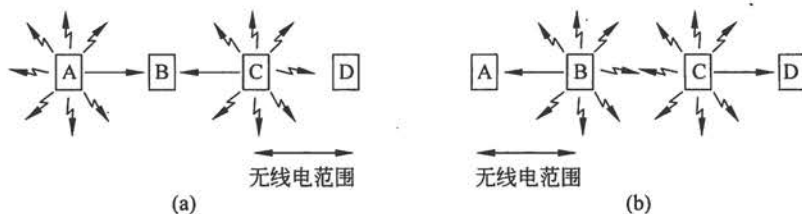


图 4-11 一个无线 LAN

(a) 在给 B 传输时 A 和 C 是隐藏终端；(b) 在给 A 和 D 传输时 B 和 C 是暴露终端

首先考虑当 A 和 C 向 B 传送数据时的情形,如图 4-11 (a) 所示。如果 A 开始发送,然后 C 立即进行侦听介质,它将不会听到 A 的传输,因为 A 在它的覆盖范围之外。因此 C 错误地得出结论:它可以向 B 传送数据。如果 C 传送数据,将在 B 处产生冲突,从而扰乱 A 发来的帧。(我们假设没有类似 CDMA 的编码模式可以提供多信道传输,因此冲突会扰乱信号,从而破坏两个帧)我们需要一个 MAC 协议,它能防止这种冲突的发生,因为冲突将导致带宽的浪费。由于竞争者离得太远而导致站无法检测到潜在的竞争者,这个问题称为**隐藏终端问题 (hidden station problem)**。

现在让我们来考虑另一种不同的情形: B 向 A 传送数据,同时 C 想给 D 发送数据,如图 4-11 (b) 所示。如果 C 侦听介质,它会听到有一个传输正在进行,从而会错误地得出结论:它不能向 D 发送数据(图中的虚线表示)。事实上, C 所听到的传输只会搞坏 B 和 C 之间区域中的接收,但是,两个接收方都不在这个危险区域。我们需要一个 MAC 协议,它能防止此类延迟传输的发生。这个问题称为**暴露终端问题 (exposed station problem)**。

这里的困难在于开始传输之前站真正希望知道的是在接收方周围是否无无线电活动。CSMA 只能告知在侦听载波的站附近是否有活动发生。对于有线情形,所有的信号能传播到全部的站,所以这种区别并不存在;然而,在同一时刻,无论在系统中任何地方都只能有一个传输在进行。在一个基于短程无线电波的系统,多个传输可以同时发生,只要它们有不同的目的地,并且这些目的地都不在彼此的范围内。我们希望当蜂窝变得越来越大时这种并发性能够发生。类似的情形发生在聚会中,舞会中的人们不会等到房间里的每个人都沉默了才开始交谈;在一个大房间里可以同时发生多个交谈,只要交谈者不在相同的位置。

能处理无线 LAN 这些问题的一个早期且有影响力的协议是**冲突避免多路访问 (MACA, Multiple Access with Collision Avoidance)** (Karn, 1990)。MACA 的基本思想是发送方刺激接收方输出一个短帧,以便其附近的站能检测到该次传输,从而避免在接下去进行的(较大)数据帧传输中也发送数据。这项技术被用来替代载波侦听。

图 4-12 说明了 MACA 协议。现在我们来考虑 A 如何向 B 发送一帧。A 首先给 B 发送一个 RTS (Request To Send) 帧,如图 4-12 (a) 所示。这个短帧 (30 字节) 包含了随后将要发送的数据帧的长度;然后, B 用一个 CTS (Clear to Send) 作为应答,如图 4-12 (b) 所示。此 CTS 帧也包含了数据长度(从 RTS 帧中复制过来)。A 在收到了 CTS 帧之后便开始传输。

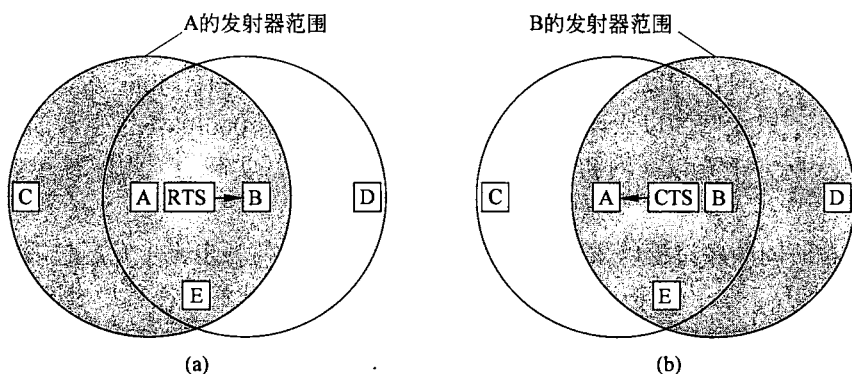


图 4-12 MACA 协议
(a) A 给 B 发送一个 RTS; (b) B 作为响应给 A 返回一个 CTS

现在我们来分析，如果其他站也听到了这些帧，它们会如何反应。如果一个站听到了 RTS 帧，那么它一定离 A 很近，它必须保持沉默，至少等待足够长的时间以便在无冲突情况下 CTS 被返回给 A。如果一个站听到了 CTS，则它一定离 B 很近，在接下来的数据传送过程中它必须一直保持沉默，只要检查 CTS 帧，该站就可以知道数据帧的长度（即数据传输要持续多久）。

在图 4-12 中，C 落在 A 的范围内，但不在 B 的范围内。因此，它听到了 A 发出的 RTS，但是没有听到 B 发出的 CTS。只要它没有干扰 CTS，那么在数据帧传送过程中，它可以自由地发送任何信息。相反，D 落在 B 的范围内，但不在 A 的范围内。它听不到 RTS 帧，但是听到了 CTS 帧。只要听到了 CTS 帧，这意味着它与一个将要接收数据帧的站离得很近；所以，它就延缓发送任何信息直到那个帧如期传送完毕。站 E 听到了这两条控制消息，与 D 一样在数据帧完成之前它必须保持安静。

尽管有了这些防范措施，冲突仍有可能发生。例如，B 和 C 可能同时给 A 发送 RTS 帧。这些帧将发生冲突，因而丢失。在发生了冲突的情况下，一个不成功的发送方（即在期望的时间间隔内没有听到 CTS）将等待一段随机的时间，以后再重试。

4.3 以太网

现在我们已经完成了关于信道分配协议的抽象讨论，是时候看看这些原则如何应用于实际系统了。许多为个域网、局域网和城域网所做的设计都已经在 IEEE 802 名义下进行了标准化。有几个协议幸免了下来，但还有许多没能逃脱被休眠的命运，正如我们在图 1-38 看到的。一些相信轮回的人甚至开玩笑说，一定是卡尔·达尔文转世加入了 IEEE 标准协会，从而淘汰那些不合时宜的标准。最重要的幸存者 802.3（以太网）和 802.11（无线局域网）。蓝牙（无线 PAN）得到了广泛部署，但现在其标准化的工作已经独立于 802.15 之外。至于 802.16（无线城域网），现在说它还过早。请参考本书第 6 版来深入了解它。

我们将开始研究以太网实际系统，这可能是现实世界中最普遍的一种计算机网络。以太网有两类：第一类是经典以太网（classic Ethernet），它使用我们在本章已学过的技术解决了多路访问问题；第二类是交换式以太网（switched Ethernet），使用了一种称为交换机的设备连接不同的计算机。重要的是要注意，虽然它们都称为以太网，但它们有很大的不同。经典以太网是指以太网的原始形式，运行速度从 3~10 Mbps 不等；而交换式以太网正是成就了以太网的以太网，可运行在 100、1000 和 10000 Mbps 那样的高速率，分别以快速以太网、千兆以太网和万兆以太网的形式呈现。实际上，现在使用的只有交换式以太网。

我们将按时间顺序讨论以太网的历史，展示它们是如何一步一步发展起来的。由于以太网和 IEEE 802.3 几乎完全一致，除了一个微小差别外（我们将在稍后讨论），许多人交替使用“以太网”和“IEEE 802.3”这两个术语。我们也将这样做。若需了解更多的以太网信息，请参见（Spurgeon, 2000）。

4.3.1 经典以太网物理层

以太网的故事始于 ALOHA 同时期,确切的时间是在一个名叫 Bob Metcalfe 的学生获得麻省理工学院的学士学位后,搬到河对岸的哈佛大学攻读博士学位之际。在他学习期间,他接触到了 Abramson 的工作,他对此很感兴趣。从哈佛毕业后,他决定在前往施乐帕洛阿尔托研究中心(Palo Alto Research Center)正式工作之前留在夏威夷度暑假,以便帮助 Abramson 工作。当他到达帕洛阿尔托研究中心,他看到那里的研究人员已经设计并建造出后来称为个人计算机的机器,但这些机器都是孤零零的;他便运用帮助 Abramson 工作获得的知识与同事 David Boggs 设计并实现了第一个局域网(Metcalfe 和 Boggs, 1976)。该局域网采用一个长的粗同轴电缆,以 3Mbps 速率运行。

他们把这个系统以发光性乙醚的名称(luminiferous ether)命名为以太网(Ethernet),人们曾经认为通过它可以传播电磁辐射(当 19 世纪英国物理学家 James Clerk Maxwell 发现电磁辐射可以用一个波方程来描述,科学家们一直认为空中充满着一种空灵的介质,电磁辐射才得以传播。只有在 1887 年著名的 Michelson-Morley 实验之后,物理学家才发现电磁辐射可以在真空中传播)。

施乐以太网(Xerox Ethernet)获得了巨大的成功,以至于 DEC、英特尔和施乐公司在 1978 年制定了一个 10Mbps 以太网标准,称为 DIX 标准(DIX standard)。做了少许修订后,DIX 标准在 1983 年正式成为 IEEE 802.3 标准。不幸的是,早已拥有一个历史性开创发明(例如个人计算机)的施乐公司后来却失败于商业化过程。《探索未来》(Fumbling the Future)告诉你一个故事(Smith 和 Alexander, 1988)。当施乐表现出对以太网除了协助制订标准外没有多大兴趣后,Metcalfe 创建了自己的公司 3Com,出售用于个人计算机的以太网适配器。它卖出了好几百万。

经典以太网用一个长电缆蜿蜒围绕着建筑物,这根电缆连接着所有的计算机。这种体系结构如图 4-13 所示。第一个产品,俗称粗以太网(thick Ethernet),用类似黄色的花园用软管连接着所有的计算机,在电缆的每 2.5 米处有个标记,这个标记就是连接计算机的位置(802.3 标准实际上并没有要求电缆是黄色,但它确实建议用黄色)。它的继任者是细以太网(thin Ethernet),电缆比粗以太网柔软,更加易于弯曲,并且使用了工业标准 BNC 接头。细以太网更便宜也更容易安装,但它单段电缆仅 185 米(而不是粗以太网的 500 米)长,且每段电缆只能处理 30 台计算机(而不是 100 台)。

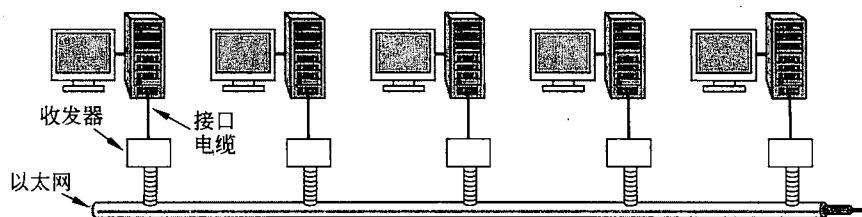


图 4-13 经典以太网体系结构

以太网的每个版本都有电缆的最大长度限制(即无须放大的长度),这个范围内的信号

可以正常传播，超过这个范围信号将无法传播。为了允许建设更大的网络，可以用**中继器**（repeater）把多条电缆连接起来。中继器是一个物理层设备，它能接收、放大（即再生）并在两个方向上重发信号。至于软件方面，一系列由中继器连接起来的电缆段与一根单独的电缆没有什么不同（除了由中继器引入的少量延迟外）。

在这些电缆上，信息的发送使用我们在 2.5 节学过的曼彻斯特编码。以太网可以包含多个电缆段和多个中继器，但是不允许任意两个收发器之间的距离超过 2.5 千米，并且任意两个收发器之间经过的中继器不能超过 4 个。之所以有这样的限制在于这是保证 MAC 协议正常工作的需要，我们下面将着眼于这点。

4.3.2 经典以太网 MAC 子层协议

帧格式如图 4-14 所示。首先是 8 个字节的前导码（Preamble），每个字节包含比特模式 10101010（除了最后一个字节的最后 2 位为 11）。这最后一个字节称为 802.3 的帧起始定界符（Start of Frame, SOF）。比特模式是由曼彻斯特编码产生的 10 MHz 方波，每个波 6.4 微秒，以便接收方的时钟与发送方同步。最后两个“1”告诉接收方即将开始一个帧。



图 4-14 帧格式
(a) 以太网（DIX）；(b) IEEE 802.3

接下来是两个地址字段，一个标识目的地址，另一个标识帧的发送方。它们均为 6 个字节长。如果传输出去的目标地址第一位是 0，则表示这是一个普通地址；如果是 1，则表示这是一个组地址。组地址允许多个站同时监听一个地址。当某个帧被发送到一个组地址，该组中的所有站都要接收它。往一组地址的发送行为称为**组播**（multicasting）。由全 1 组成的特殊地址保留用作**广播**（broadcasting）。如果一个帧的目标地址字段为全 1，则它被网络上的所有站接收。组播是更多的选择，但它涉及确定组内有哪些成员的组管理。相反，广播根本不区分站，因此不需要任何组管理机制。

站的源地址有一个有趣的特点，那就是它们具有全球唯一性。该地址由 IEEE 统一分配，因此确保了在世界任何地方没有两个站的地址是相同的。只要给出正确的 48 位数字，任何站都可以唯一寻址到该数字代表的任何其他站。要做到这一点，地址字段的前 3 个字节用作该站所在的**组织唯一标识符**（OUI, Organizationally Unique Identifier）。该字段的值由 IEEE 分配，指明了网络设备制造商。制造商获得一块大小为 2^{24} 的地址。地址字段的最后 3 个字节由制造商负责分配，并在设备出厂之前把完整的地址用程序编入 NIC。

接下来是类型（Type）或长度（Length）字段，究竟采用哪个字段取决于以太网帧还是 IEEE 802.3 帧。以太网使用类型字段告诉接收方帧内包含了什么。同一时间在同一台机器上或许用了多种网络层协议，所以当以一个以太帧到达接收方时，操作系统需要知道应

该调用哪个网络层协议来处理帧携带的数据包。类型字段指定了把帧送给哪个进程处理。例如，一个值为 0x0800 的类型代码意味着帧内包含一个 IPv4 的数据包。

IEEE 802.3 以其智慧决定该字段携带帧的长度，因为以太网的长度必须由其内部携带的数据来确定——如果真是这样的话，则违反了分层规定。当然，IEEE 的处理方式意味着接收方没有办法确定如何处理入境帧。这个问题由数据内包含的另一个逻辑链路控制 (LLC, Logical Link Control) 协议头来处理。它使用 8 个字节来传达 2 个字节的协议类型信息。

不幸的是，在 802.3 标准出炉时，已经有太多的 DIX 以太网硬件和软件在使用，以至于很少有厂家和客户有热情来重新包装类型和长度字段。1997 年，IEEE 认输并表示这两种字段都可以接受。幸运的是，所有在 1997 年之前使用的类型字段其值都大于 1500，因此规定了最大数据长度。现在的规则是，任何值小于或等于 0x600 (1536) 可解释为长度字段，任何大于 0x600 可解释为类型字段。现在 IEEE 可以认为每个人都使用了它的标准，并且其他人可以继续做他们正在做的事情（不能打扰 LLC），它因而无须感到内疚。

接下来是数据 (Data) 字段，最多可包含 1500 个字节。在制定 DIX 标准时，这个值的选择有一定的随意性。当时最主要的考虑是收发器需要足够的内存 (RAM) 来存放一个完整的帧，而 RAM 在 1978 年时还很昂贵。这个上界值越大意味着需要的 RAM 更多，因而收发器的造价更高。

除了有最大帧长限制外，还存在一个最小帧长的限制。虽然有时候 0 字节的数据字段也是有用的，但它会带来一个问题。当一个收发器检测到冲突时，它会截断当前的帧，这意味着冲突帧中已经送出的位将会出现在电缆上。为了更加容易地区分有效帧和垃圾数据，以太网要求有效帧必须至少 64 字节长，从目标地址算起直到校验和，包括这两个字段本身在内。如果帧的数据部分少于 46 个字节，则使用填充 (Pad) 字段来填充该帧，使其达到最小长度要求。

限制最小帧长的另一个（也是更重要的）理由是避免出现这样的情况：当一个短帧还没有到达电缆远端的发送方，该帧的传送就已经结束；而在电缆的远端，该帧可能与另一帧发生冲突。这个问题如图 4-15 所示。在 0 时刻，位于电缆一端的站 A 发出一帧。我们假设该帧到达另一端的传播时间为 τ 。正好在该帧到达另一端之前的某一时刻（即，在 $\tau - \epsilon$ 时刻），位于最远处的站 B 开始传送数据。当 B 检测到它所接收到的信号比它发送的信号更强时，它知道已经发生了冲突，所以放弃了自己传送，并且产生一个 48 位的突发噪声以

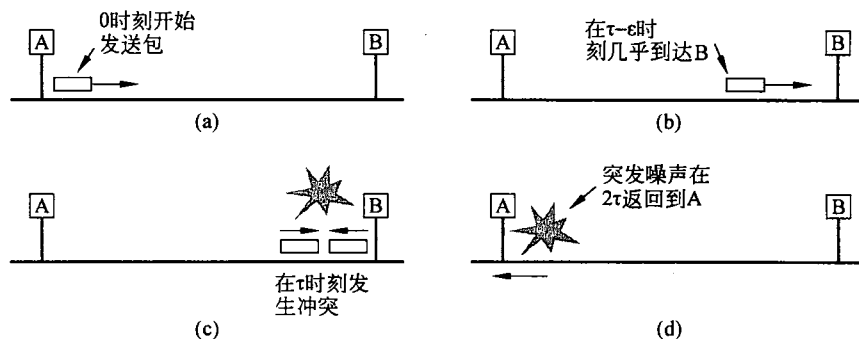


图 4-15 冲突检测只需要 2τ 时间

警告所有其他站。换句话说，它阻塞了以太网，以便确保发送方不会漏检这次冲突。大约在 2τ 后，发送方看到了突发噪声，并且也放弃自己的传送。然后它等待一段随机的时间，再次重试。

如果一个站试图传送非常短的帧，则可以想象：虽然发生了冲突，但是在突发噪声回到发送方（ 2τ ）之前，传送已经结束。然后，发送方将会得出刚才一帧已经成功发送的错误结论。为了避免发生这样的情况，所有帧必须至少需要 2τ 时间才能完成发送，这样当突发噪声回到发送方时传送过程仍在进行。对于一个最大长度为 2500 米、具有 4 个中继器的 10 Mbps LAN（符合 802.3 规范），在最差情况下，往返一次的时间大约是 50 微秒（其中包括了通过 4 个中继器所需要的时间）。因此，允许的最小帧长必须至少需要这样长的时间来传输。以 10 Mbps 的速率，发送一位需要 100 纳秒，所以 500 位是保证可以工作的最小帧长。考虑到加上安全余量，该值被增加到 512 位，或者 64 字节。

最后一个字段是校验和（Checksum）。它是我们在 3.2 节学过的 32 位 CRC。事实上，它由我们曾经给出的生成多项式确切定义，这个 CRC 同样被用于 PPP、ADSL 和其他链路层协议。CRC 是差错检测码，用来确定接收到的帧比特是否正确。它只提供检错功能，如果检测到一个错误，则丢弃帧。

二进制指数后退的 CSMA/CA

经典以太网使用 1-坚持 CSMA/CD 算法，我们在 4.2 节对此有所描述。这个算法意味着当站有帧需要发送时要侦听介质，一旦介质变为空闲便立即发送。在它们发送的同时监测信道上是否有冲突。如果有冲突，则立即中止传输，并发出一个短冲突加强信号，在等待一段随机时间后重发。

现在我们来了解当冲突发生后如何确定随机等待的时间。我们仍然使用图 4-5 所示的模型。在冲突发生后，时间被分成离散的时间槽，其长度等于最差情况下在以太介质上往返传播时间（ 2τ ）。为了达到以太网允许的最长路径，时间槽的长度被设置为 512 比特时间，或 51.2 微秒。

第一次冲突发生后，每个站随机等待 0 个或者 1 个时间槽，之后再重试发送。如果两个站冲突之后选择了同一个随机数，那么它们将再次冲突。在第二次冲突后，每个站随机选择 0、1、2 或者 3，然后等待这么多个时间槽。如果第三次冲突又发生了（发生的概率为 0.25），则下一次等待的时间槽数从 0 到 2^3-1 之间随机选择。

一般地，在第 i 次冲突之后，从 $0 \sim 2^i-1$ 之间随机选择一个数，然后等待这么多个时间槽。然而，达到 10 次冲突之后，随机数的选择区间被固定在最大值 1023，以后不再增加。在 16 次冲突之后，控制器放弃努力，并给计算机返回一个失败报告。进一步的恢复工作由高层协议完成。

这个算法称为二进制指数后退（binary exponential backoff），它可以动态地适应发送站的数量。如果所有冲突的随机数最大值都是 1023，则两个站第二次发生冲突的概率几乎可以忽略；但是，在一次冲突之后的平均等待时间将是数百个时间槽，这样会引入很大的延迟。另一方面，如果有 100 个站都要发送数据，每个站总是等待 0 个或者 1 个时间槽，那么，它们将一而再、再而三地一次次发生冲突，直到其中的 99 个站选择 1 而剩下一个站选择 0。这种情况可能需要等上好几年的时间才有可能发生。随着连续冲突的次数越来越多，

随机等待的间隔呈指数增加, 这样算法能确保两种情况: 如果只有少量站发生冲突, 则它可确保较低的延迟; 当许多站发生冲突时, 它也可以保证在一个相对合理的时间间隔内解决冲突。将延迟后退的步子截断在 1023 可避免延迟增长得太大。

如果没有发生碰撞, 发送方就假设该帧可能被成功传递了。也就是说, 无论是 CSMA/CD 还是以太网都不提供确认。这样的选择适用于出错率很低的有线电视和光纤信道。确实发生的任何错误必须通过 CRC 检测出来并由高层负责恢复。对于无线信道, 因其出错率高还得使用确认手段, 这点我们将在后面章节部分说明。

4.3.3 以太网性能

现在, 让我们简要地探讨在重负载和恒定负载条件下 (即总是有 k 个站要传送数据) 经典以太网的性能。关于二进制指数后退算法的严格分析非常复杂。因此, 我们采用了 (Metcalfe 和 Boggs, 1976) 方法, 并假定每个时间槽中重传的概率是个常数。如果每个站在一个竞争时间槽中传送帧的概率为 p , 那么, 在这个时间槽中, 某一个站获得信道的概率 A 为:

$$A = kp(1-p)^{k-1} \quad (4-5)$$

当 $p=1/k$ 的时候, A 最大; 并且当 $k \rightarrow \infty$ 的时候, $A \rightarrow 1/e$ 。竞争间隔正好等于 j 个时间槽的概率为 $A(1-A)^{j-1}$, 所以每一次竞争的平均时间槽数为:

$$\sum_{j=0}^{\infty} jA(1-A)^{j-1} = \frac{1}{A}$$

由于每个时间槽的间隔时间为 2τ , 因此平均竞争间隔 w 为 $2\tau/A$ 。假设最优 p , 并且竞争时间槽的平均数永远不超过 e , 于是, w 至多为 $2\tau e \approx 5.4\tau$ 。

如果传送一帧平均需要 P 秒, 那么, 当许多站都要传送帧时, 信道效率

$$\text{信道效率} = \frac{P}{P + 2\tau/A} \quad (4-6)$$

这里我们可以看到任何两个站之间的最大电缆距离也会影响到性能。电缆越长, 则竞争间隔也越长。这正是为什么以太网标准规定了最大电缆长度的原因。

针对每个帧 e 个竞争时间槽的最优情形, 按照帧的长度 F 、网络带宽 B 、电缆长度 L 和信号的传播速度 c , 来制定等式 (4-6) 是有指导意义的。利用 $P = F/B$, 等式 (4-6) 变成:

$$\text{信道效率} = \frac{1}{1 + 2BLE/cF} \quad (4-7)$$

当分母中的第二项变大时, 网络的效率将会变低。特别是, 在给定帧长度的情况下, 增加网络带宽或者距离 (即 BL 的乘积) 将会降低网络效率。不幸的是, 许多网络硬件方面的研究都要把增大此乘积值作为目标。人们总是希望在长距离上拥有高带宽 (例如, 光纤城域网), 而用这种方式实现的以太网可能并不是这些应用的最佳系统。我们将在下节看到实现以太网的其他一些方法。

图 4-16 给出了利用等式 (4-7), 在 $2\tau=51.2$ 微秒以及 10 Mbps 数据速率的情况下, 信道效率与就绪站数之间的关系。对于 64 字节的时间槽时间, 64 字节的帧并不是最有效的, 这并不奇怪。另一方面, 当帧长度为 1024 字节, 以及每个竞争间隔趋近于 e 个 64 字节时

间槽时，竞争期为 174 字节长，效率为 85%。这个结果比分槽 ALOHA 的 37% 利用率要好得多。

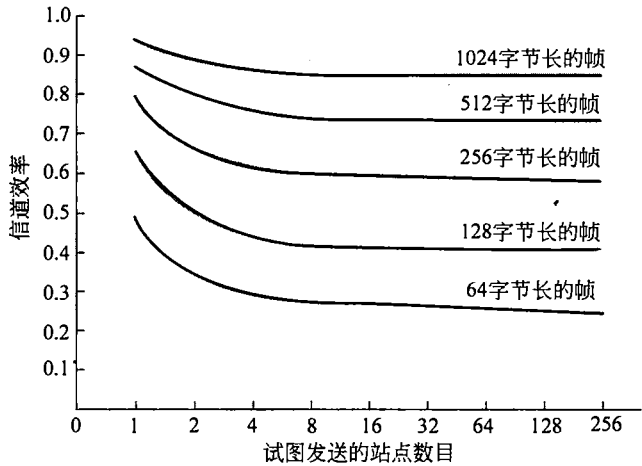


图 4-16 具有 512 位时间槽的 10Mbps 以太网效率

或许有一点值得一提，关于以太网（和其他的网络）有大量理论性的性能分析成果。大多数研究结果都只有很少的参考价值，原因有两个。首先，几乎所有理论工作都假设网络流量符合泊松分布。当研究人员开始观察实际数据，他们发现网络流量很少呈现泊松分布，而是在一定的时间尺度上表现出自相似或者突发性（Paxson 和 Floyd, 1995; Leland 等, 1994）。这意味着即使在一段较长的时间上进行平均也不能使流量变得平滑。除了理论分析时采用的可疑模型外，许多研究把分析重点放在异常高负载情况下的“有趣”性能上。（Boggs 等, 1988）通过实验显示以太网在实际情况下工作很好，即使在适度的高负载下。

4.3.4 交换式以太网

以太网的发展很快，从单根长电缆的典型以太网结构开始演变。单根电缆存在的问题，比如找出断裂或者松动位置等与连接相关的问题，趋势人们开发出一种不同类型的布线模式。在这种模式中，每个站都有一条专用电线连接到一个中央集线器。集线器只是在电气上简单地连接所有连接线，就像把它们焊接在一起。这种配置如图 4-17（a）所示。

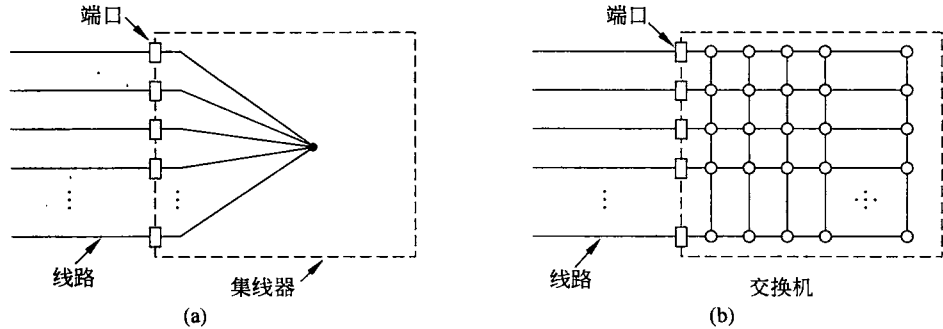


图 4-17
(a) 集线器；(b) 交换机

电线就是电话公司的双绞线，因为大多数写字楼已布有这种线，而且通常足够富裕。电话线的这种重用是个双赢局面，但它把允许的最长电缆长度减少到目前离集线器不得大于 100 米的限制（如果是高品质的 5 类双绞线则可扩展到 200 米）。在这种配置下，添加或删除一个站非常容易，电缆断裂也很容易被直接检测出来。因为具有使用现有布线和易于维护等优点，双绞线集线器迅速成为以太网的主要形式。

然而，集线器不能增加容量，因为它们逻辑上等同于单根电缆的经典以太网。随着越来越多的站加入，每个站获得的固定容量共享份额下降。最终，LAN 将饱和。一条出路是提升到更高的速度，比如从 10 Mbps 升到 100 Mbps、1 Gbps，甚至更高的速率。但是，随着多媒体和功能强大服务器的增长，即使 1 Gbps 以太网也会变得饱和。

幸运的是，还有另一条出路可以处理不断增长的负载：即交换式以太网。这种系统的核心是一个交换机（switch），它包含一块连接所有端口的高速背板，如图 4-17（b）所示。从外面看，交换机很像集线器。它们都是一个盒子，通常拥有 4~48 个端口，每个端口都有一个标准的 RJ-45 连接器用来连接双绞线电缆。每根电缆把交换机或者集线器与一台计算机连接，如图 4-18 所示。交换机具有集线器同样的优点。通过简单的插入或者拔出电缆就能完成增加或者删除一台机器，而且由于片状电缆或者端口通常只影响到一台机器，因此大多数错误都很容易被发现。这种配置模式仍然存在一个共享组件出现故障的问题，即交换机本身的故障；如果所有站都失去了网络连接，则 IT 人员知道该怎么解决这个问题：更换整个交换机。

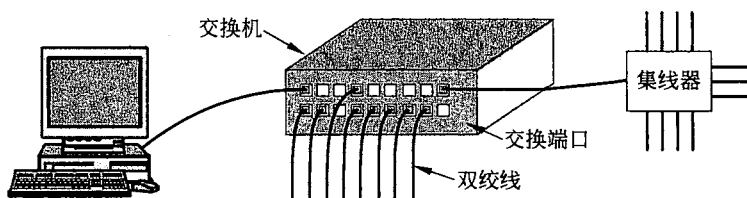


图 4-18 一个以太网交换机

然而，进入交换机内部就能看到某些与集线器完全不同的事情。交换机只把帧输出到该帧想去的端口。当交换机端口接收到来自某个站的以太网帧，它就检查该帧的以太网地址，确定该帧前往的目的地端口。这一步要求交换机能够知道端口对应哪个地址，这个过程我们放在 4.8 节学习了交换机之间互连的一般情况后再来讨论。现在，我们只假设交换机知道帧的目标端口。交换机把帧通过它的高速背板传送到目标端口。通常背板的运行速度高达许多个 Gbps，并且使用专用的协议。这些交换机专用协议无须标准化，因为它们完全隐藏在交换机内部；然后，目标端口在通往目标站的双绞线上传输该帧，没有任何其他端口知道这个帧的存在。

如果同时有多个站或者端口都要传送数据，会发生什么情况？在这点上，交换机再一次体现了与集线器的不同。在集线器中，所有站都位于同一个冲突域（collision domain），它们必须使用 CSMA/CD 算法来调度各自的传输。在交换机中，每个端口有自己独立的冲突域。通常情况下，电缆是全双工的，站和端口可以同时往电缆上发送帧，根本无须担心其他站或者端口。现在冲突不可能发生，因而 CSMA/CD 也就不需要了。然后，如果电缆是半双工的，则站和端口必须以通常的 CSMA/CD 方式竞争传输。

交换机的性能优于集线器有两方面的原因。首先，由于没有冲突，容量的使用更为有效。其次，也是更重要的，有了交换机可以同时发送多个帧（由不同的站发出）。这些帧到达交换机端口并穿过交换机背板输出到适当的端口。然而，由于两帧可能在同一时间去往同一个输出端口，交换机必须有缓冲，以便它暂时把输入帧排入队列直到帧被传输到输出端口。总体而言，这些改进获得了较大的性能改进，而这些改进手段对于集线器来说又是不可能做到的。系统总吞吐量通常可以提高一个数量级，主要取决于端口数目和流量模式。

帧被发送到输出端口还有利于安全。大多数 LAN 接口都支持混杂模式（promiscuous mode），这个模式下所有的帧都被发到每台计算机，而不只是那些它寻址的机器。每个连到集线器上的计算机能看到其他所有计算机之间的流量。间谍和好管闲事者最喜欢这一功能了。有了交换机，流量只被转发到它的目的端口。这种限制提供了更好的隔离措施，使得流量不会轻易泄露而落入坏人手中。不过，如果真正需要安全，最好还是对流量实行加密。

由于交换机只是希望每个输入端口出现的是标准以太网帧，就有可能把一部分端口用作集中器。在图 4-18 中，右上角的端口并没有连接一台计算机，而是连到一个 12 口的集线器。当帧到达集线器，它们按通常的方式竞争以太网，包括冲突和二进制指数后退。竞争成功的帧被传到集线器，继而到达交换机，在那里得到的处理就像任何其他输入帧一样。交换机不知道它们是经过竞争才到这里的。一旦进入交换机，它们就被发送到高速背板上的正确输出线。也有可能正确目的地是连接到集线器的一根线，在这种情况下，帧早已经被交付给目标机器，因此交换机就把它丢弃。集线器比交换机简单而且便宜，但由于交换机的价格在不断下降，集线器已经成为濒危物种。现代网络大量使用交换以太网。然而，传统的集线器依然存在。

4.3.5 快速以太网

在同一时间，交换机变得越来越受欢迎，10 Mbps 以太网备受压力。起初，10 Mbps 速率似乎完美无缺，如同线缆调制解调器在电话调制解调器用户看来像天堂一样。但是，新鲜感很快消失。作为帕金森定律（“工作总会填满原来计划用的时间”）的一种必然结果，似乎数据传输也总是会占用所有的可用带宽。

许多安装需要大量的带宽，因而许多 10 Mbps 的局域网被中继器、集线器和交换机连成了迷宫；虽然对网络管理员而言，有时他们觉得自己和泡泡糖和铁丝网挤在一起。但即使有以太网交换机，一台计算机的最大带宽还是受制于连接它到交换机端口的电缆。

在这种环境下，IEEE 于 1992 年重新召集 802.3 委员会，指示他们赶快提出一个快速 LAN 建议。其中一个建议是仍然保持 802.3 原来的面貌不变，但要运行得更快。另一个建议是完全重新设计 802.3，给予它更多的特性，比如支持实时流量和数字化语音，但是仍保留原来的名称（出于市场考虑的原因）。经过多次激烈争论之后，802.3 委员会决定仍然保留 802.3 原来的工作方式，但是让它运行得更快。这一策略将使标准化工作得以在技术革新之前完成，并且避免了全新设计带来的不可预见问题。新设计还将向后兼容现有的以太网局域网。在这种情况下那些提议遭到否决的人如同自尊的计算机工业界习惯做法那样：他们跺着脚形成了自己的委员会并标准化他们提议的局域网（最终作为 802.12）。但他们还

是以失败告终，草草收场。

这项工作很快就完成了（按照标准委员会的规范），其结果就是 802.3u，于 1995 年被 IEEE 正式批准。技术上，802.3u 并不是一个新标准，而是原有 802.3 标准的一份补充（因而也加强了它的向后兼容性）。这种策略被采用了许多次。因为实际上大家都称它为快速以太网（fast Ethernet），而不是 802.3u，所以我们将使用这样的叫法。

快速以太网的基本思想非常简单：保留原来的帧格式、接口和过程规则，只是将比特时间从 100 纳秒降低到 10 纳秒。技术上，它可以照搬 10 Mbps 的经典以太网，只要将电缆的最大长度降低到十分之一，以便及时检测冲突。然而，由于双绞线具有压倒性的优势，所以快速以太网基本上完全基于这种设计。因此，所有的快速以太网系统也使用集线器和交换机；而不允许使用带插入式分接头或者 BNC 连接器的多支路电缆。

然而，还有其他一些选择仍然要确定下来，其中最重要的是支持什么样的电缆类型。一种观点是 3 类双绞线，其理由是几乎西方国家的每个办公室都有至少 4 组 3 类（或更好的）双绞线，从办公室连接到 100 米以内的电话接线柜。有时候会有两条这样的电缆。因此，若采用 3 类双绞线，则无须重新布线就有可能在桌面计算机使用快速以太网，这对于许多组织来说具有极大的好处。

使用 3 类双绞线的主要缺点是它不能够在 100 米长的电缆上承载 100 Mbps 的信号，而这是 10 Mbps 集线器规定的从计算机到集线器的最大距离。相反，5 类双绞线可以很容易地传输 100 米，光纤可以走得更远。最后折中的选择是允许所有这三种可能介质，如图 4-19 所示，但是，对于 3 类双绞线的方案，需要增加所需的额外承载能力。

名称	线缆	最大长度	优点
100Base-T4	双绞线	100 米	可用 3 类 UTP
100Base-TX	双绞线	100 米	全双工速率 100 Mbps (5 类 UTP)
100Base-FX	光纤	2000 米	全双工速率 100 Mbps，距离长

图 4-19 最初的快速以太网线

3 类 UTP 方案称为 100Base-T4，它使用了 25 MHz 的信令速度，比标准以太网的 20 MHz 仅仅快了 25%（记住，2.5 节中讨论的曼彻斯特编码中，对于 10 Mbps 中的每个比特要求两个时钟周期）。然而，为了达到所要求的比特率，100Base-T4 要求 4 对双绞线。其中一对总是去往集线器，另一对总是来自集线器，剩余的两对则动态切换到当前的传输方向。为了从传输方向上的三对双绞线上获得 100 Mbps，每对双绞线上使用了一种相当复杂的编码方案。该方案涉及用三个不同电压等级来发送三元数字。这个方案不太可能赢得任何奖项，我们将跳过细节。然而，由于过去几十年来标准的电话线每根电缆中都有 4 对双绞线，所以大多数办公室都能够使用已有的布线。当然，这也意味着要放弃你的办公室电话。但是，相对于能获得快速的电子邮件所付出的这点代价应该算不了什么。

随着许多办公大楼重新布线采用了 5 类 UTP，100Base-T4 也随之落下了帷幕。使用 5 类 UTP 的 100Base-TX 以太网很快占领了市场。因为 5 类双绞线可以处理 125 MHz 的时钟速率，所以这种设计要简单得多。每个站只用到两对双绞线，一对用于发送信号到集线器，另一对用于从集线器接收信号。这里没有使用直接的二进制编码（即 NRZ），也没有采用曼彻斯特编码，相反，它使用了一种我们在 2.5 节中描述过的 4B/5B 编码方案。4 个数据

位被编码成 5 个信号比特, 并以 125 MHz 速率发送, 可提供 100 Mbps 的数据率。这种方案很简单, 但具有保持同步所需的足够跳变, 对线缆带宽的使用相对很好。100Base-TX 系统是全双工的; 站可以在一对双绞线上以 100 Mbps 速率发送数据, 同时也可以在这一对双绞线上以 100 Mbps 速率接收数据。

最后一种选择方案是 100Base-FX, 它使用两根多模光纤, 每个方向用一根; 所以它可以进行全双工操作, 同时每个方向上以 100 Mbps 速率发送。这种设置下, 站和交换机之间的距离可以达到 2 千米。

快速以太网允许集线器和交换机相互之间互连。为了确保 CSMA/CD 算法继续工作, 为了把网络传输速率从 10 Mbps 提高到 100 Mbps, 必须保持最小帧长和最大电缆长度之间的关系。所以, 要么按比例把最小帧为 64 字节大小的限制往上提, 要么把 2500 米的最大电缆长度降下来。最简单的选择是把任意两个站之间的最大距离降 10 倍, 因为 100 米电缆的集线器早已可用。然而, 2 千米 100Base-FX 的线缆对于正常的以太网冲突算法来说过长。相反, 这些线缆必须被连接到一个交换机, 并工作在全双工模式下, 因此才没有冲突。

用户很快就开始部署了快速以太网, 但他们并不打算扔掉老式计算机上的 10 Mbps 以太网卡。因此, 几乎所有的快速以太网交换机可处理 10 Mbps 和 100 Mbps 的混合情况。为了便于升级, 标准本身提供了一个称为自动协商 (autonegotiation) 的机制, 允许两个站自动协商最佳速度 (10 Mbps 或 100 Mbps) 和双工模式 (半双工或全双工)。这个机制大部分时间运作良好, 但已经发现会导致双工不匹配问题, 即链路的一端启动了自动协商, 但另一端没有启动, 并设置了全双工模式 (Shalunov 和 Carlson, 2005)。大多数以太网产品都使用该功能来配置自己。

4.3.6 千兆以太网

快速以太网标准的墨迹未干, 802 委员会就开始制定一项更快的快速以太网, 称为千兆以太网 (gigabit Ethernet)。1999 年 IEEE 批准了最普遍的形式 802.3ab。下面我们将讨论千兆以太网的一些关键特性。更多信息请参考 (Spurgeon, 2000)。

对于千兆位以太网, 委员会的目标基本上与快速以太网的目标相同: 增加十倍的性能, 并同时保持与所有现存以太网标准的兼容。特别是, 千兆以太网必须提供单播和广播的无确认数据报服务, 使用相同的 48 位地址方案, 保持相同的帧格式, 包括最小和最大帧尺寸要求。最终发布的标准符合所有这些目标。

与快速以太网一样, 千兆以太网使用点到点链路。最简单的配置, 如图 4-20 (a) 所示, 两台计算机直接相连。然而, 较常见的情况下, 使用一个交换机或集线器连接多台计算机, 和额外的交换机或集线器, 如图 4-20 (b) 所示。在这两种配置下, 每根单独的以太网电缆上正好有两个设备, 一个也不多一个也不少。

千兆以太网支持两种不同的操作模式: 全双工模式和半双工模式。“正常”模式是全双工模式, 它允许两个方向上的流量同时进行。这种模式适用于一台中心交换机将周围的计算机 (或者其他的交换机) 连接起来。在这种配置下, 所有的线路都具有缓存能力, 所以每台计算机或者交换机在任何时候都可以自由地发送帧。发送方在发送之前不必侦听信道才能确定是否有别人在使用信道, 因为竞争不可能发生。在一台计算机与交换机之间的连